

Temario de Sistemas operativos Parcial 1

Parte Teórica

Tema I Introducción

Funciones del Sistema Operativo

“Un programa que administra todos los otros programas en el computador”, además de los recursos con los que cuenta el ordenador

Tiene 2 funciones principales y son:

Administrar los recursos físicos:

- Manejar varios dispositivos: entre ellos se encuentran los siguientes, procesador, memoria, discos, redes, pantallas, cámaras, entre otros. De forma eficiente, confiable tolerando y enmascarando fallas.

Proveer un ambiente de ejecución para las aplicaciones corriendo sobre el computador (programas como Word, VSCode, Firefox, entre otros.):

- Provee recursos virtuales y sus interfaces: archivos, directorios, threads, procesos, entre otros. Simplifica la programación mediante la abstracción de alto nivel. Provee al usuario un ambiente estable.

Estructuras

¿Que estructura interna tiene un sistema operativo?

Los Sistemas operativos de propósito general son programas muy extensos.

Estructura simple o monolítica: MS-DOS

- Escrito para proveer la mayor cantidad de funcionalidades en el menor espacio posible. No está dividido en módulos.
- Aunque MS-DOS posee una estructura, sus interfaces y funcionalidades no están claramente separadas, dispositivos de mano: móviles y tabletas

UNIX

- Limitado por las funcionalidades de hardware, el UNIX original poseía una estructura limitada.
- El sistema operativo UNIX se estructuraba en dos partes separadas:
 - Los programas del sistema
 - El kernel: Consiste en todo lo que reside por debajo de la interfaz de llamadas al sistema y por encima del hardware. Provee sistemas de

archivos, planificación de CPU, manejo de memoria y otras funciones del so, de hecho una gran cantidad de funcionalidades para un solo nivel.

Por capas

El sistema operativo esta dividido en un numero de capas (niveles), cada una encima de la otra. La capa mas baja (nivel 0) es el hardware, mientras que la capa más alta (nivel n) es la interfaz de usuario. Haciendo uso de la modularidad producto de este enfoque, las capas son seleccionadas, así como las funciones y servicios que ellas ofrecen a sus superiores.

MicroKernel

Características:

La idea es mover una gran cantidad de funcionalidades al espacio de usuario.

¿Por que?

La comunicación se lleva a cabo mediante el pase de mensajes entre los módulos que residen en el espacio de usuario.

Beneficios:

- Es sencillo de extender las funcionalidades del MicroKernel.
- Es fácil portar el Sistema Operativo a nuevas arquitecturas.
- Mayor confiabilidad.
- Menos código ejecutarse en el espacio del kernel.
- Mayor seguridad.
- Desventajas: Sobrecarga ocasionada por la constante comunicación entre los módulos que se ejecutan en el espacio de usuario.

Modulos

- Los Sistemas Operativos modernos implementan el soporte de carga de módulos kernel.
- Usan un enfoque orientado a objetos.
- Cada componente principal se encuentra separado.
- La comunicación se realiza mediante interfaces bien conocidas.
- Es posible cargar o descargar funcionalidades del kernel en caliente.
- Su arquitectura es muy similar a la estructura de capas, pero más flexible.

Sistemas Híbridos

- La mayoría de los sistemas operativos modernos no implementa un modelo puro
- Los modelos híbridos combinan diferentes enfoques con la finalidad de mejorar en desempeño, seguridad, entre otros.

- Linux y Solaris utilizan un enfoque monolítico con soporte de módulos
- Windows es básicamente monolítico, con una estructura de MicroKernel para diferentes subsistemas
- Mac OS x utiliza un enfoque híbrido de capas, una interfaz gráfica (Aqua), mas un ambiente de programación (Cocoa)
- Por debajo, el kernel se estructura como un microkernel estilo Mach y un conjunto de partes de UNIX BSD. Adicionalmente, el kernel soporta la carga y descarga de módulos, funcionalidad conocida como soporte de extensiones de kernel y extensiones de I/O

Android

Desarrollada por google en alianza con la Open Handset Alliance

Código abierto

- Estructurado en capas
- Basado en el kernel de Linux, pero con modificaciones importantes.
- Provee manejo de procesos, memoria, drivers de dispositivos.
- Mejoras en el manejo de energía
- El ambiente de ejecución incluye un conjunto de bibliotecas y la maquina virtual Dalvik

Tipos de Sistemas Operativos

Procesamiento por lotes

- Fueron los primeros Sistemas Operativos
- Objetivo: automatizar la ejecución de trabajos y aumentar la utilización del procesador
- Los trabajos se agrupan en lotes que se iban ejecutando en secuencia
- Los más primitivos era secuenciales, la multiprogramacion se incorporo a finales de los 50
- Primer lenguaje para dictar tareas al Sistema Operativo JCL (Job Control Language)

Sistemas de tiempo compartido

- Inventados en los 1950, comercializados en los 60
- Avances sobre los sistemas por lotes, para conseguir interactividad
- Cada proceso dispone de una pequeña rodaja de tiempo periódica
- Los procesos se van turnando en la CPU
- Si la rodaja de tiempo es lo bastante pequeña (milisegundos), el usuario no percibe las pausas periódicas de su sesión

Sistemas de tiempo real

- Diseñados para cumplir tareas que deben completarse en un plazo prefijado

- Usan algoritmos de planificación de procesador especiales
- S.T.R. Critico: para la industria y sistemas empujado en los que el cumplimiento de plazos es critico. Suelen prescindir de servicios que afectan a los tiempos.

Sistemas Multiusuario

- Un sistema multiusuario reconoce que hay varios perfiles de acceso, con privilegios distintos:
 - Permisos de acceso a ficheros y aplicaciones
 - Cuotas de espacio o de tiempo de procesador
 - Prioridad en el acceso a los recursos
- Cabe destacar que los sistemas multiusuario son diferentes de los multitarea

Multiprocesadores

- Maneja de uno a varios procesadores
- Varios modelos de acceso a memoria:
 - UMA: Memoria compartida
 - NUMA: Memoria compartida
- Varios modelos de ejecución de procesos:
 - SMP (Multiprocesamiento simétrico): una tarea se puede ejecutar en cualquier procesador
 - AMP (Multiprocesamiento asimétrico): Hay especialización de tareas

Sistemas Distribuidos

- Consiste en un conjunto de computadores conectados en red y que se utilizan como si fuera un único sistema con múltiples procesadores + una gran memoria compartida + un gran almacenamiento secundario
- No existe un <Sistema distribuido universal>, pero si hay servicios con características de sistemas distribuidos (Servicios en la nube como dropbox, amazon y la WWW).

Maquinas Virtuales

- Emulación por software de una maquina física.
- Sobre la maquina virtual pueden ejecutarse programas implementados para la maquina física emulada
- No necesita el sistema original
- Le ejecución es mas lenta

Uso de las maquinas virtuales:

- Crear entornos protegidos: cada maquina virtual esta aislada de las otras.

- Independencia de la plataforma, emulación por software de una maquina fisica
- Superveniencia de sistemas antiguos
- Se pueden escribir y probar aplicaciones para un hardware que no tenemos. Sobre la maquina virtual pueden ejecutarse programas implementados para la maquina fisica emulada

Tema II-A Planificación de Procesos

Definición de proceso

Desde el punto de vista del Sistema Operativo es la entidad más pequeña, individualmente planificable, formada por código (instrucciones de máquina y llamadas al sistema) y datos; caracterizada por atributos (asignados por el programador del sistema o por el S.O. tales como prioridad, derechos de acceso, entre otros) y un estado dinámico (inactivo, listo, en ejecución, espera o completado), cada proceso tiene su program counter, y avanza cuando el proceso tiene asignado el recurso procesador. A su vez, a cada proceso se le asigna un número que lo identifica entre los demás: identificador de proceso (process id).

Un proceso en memoria se constituye de varias secciones:

Código(text): Instrucciones del proceso.

Datos(data): Variables globales del proceso.

Memoria dinámica (Heap): Memoria dinámica que genera el proceso.

Pila (Stack): Utilizado para preservar el estado en la invocación anidada de procedimientos y funciones.

Representación de los procesos (PCB)

Es una estructura de datos en la que el S.O. agrupa toda la información que necesita conocer respecto a un proceso en particular. Cada vez que se crea un proceso el S.O. crea una PCB y cuando el proceso termina su PCB es liberada.

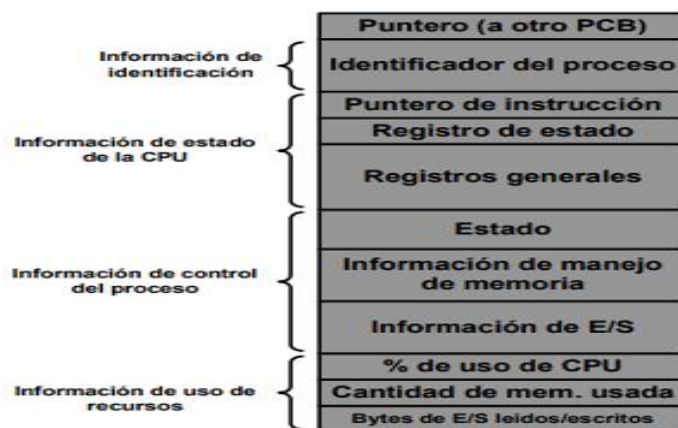


Diagrama de Estados de los procesos (5 y 7 estados)

- ***Qué pasa en cada estado y cómo un proceso pasa de un estado a otro***

Diagrama de 7 estados



¿Que pasa en cada estado?

Nuevo(new): Cuando el proceso es creado

Listo (ready): El proceso está listo para ejecutar, solo necesita del recurso procesador

Ejecutando (running): El proceso tiene asignado un procesador y está ejecutando sus instrucciones

Bloqueado (waiting): El proceso está esperando por un evento (que se complete un pedido de E/S o una señal)

Listo suspendido: Está en memoria secundaria esperando un evento

Bloqueado suspendido: Está en memoria secundaria disponible para ejecutarse

Finalizado (terminated): El proceso finalizó su ejecución

Transición de estados

Nuevo - listo: El proceso ya esta preparado para su ejecución y hay espacio en la memoria principal.

Nuevo - listo/suspendido: Esta en espera que se desocupe la memoria principal, para pasar a la cola de listos y así ser ejecutado.

Listo suspendido - Listo: Si no hay procesos listos, necesitamos traer uno para ejecutarlo. También puede que un proceso listo suspendido tenga la mayor prioridad.

Listo - Listo suspendido: Normalmente se suspenden procesos bloqueados, pero se puede preferir suspender un proceso listo de baja prioridad ante uno bloqueado de alta prioridad. Se busca obtener más memoria libre.

Listo - Ejecución: proceso que le llega su turno y pasa a a ser ejecutado.

Ejecutando - Listo/Suspendido: Un proceso de mayor prioridad despierta de bloqueado suspendido.

Ejecutando - listo: Se le acabo el quantum de tiempo al proceso, por ende se pasa a la cola de listos nuevamente.

Ejecutando - terminado: proceso que se ejecuto hasta el final.

Ejecutando - bloqueado: el proceso no puede ejecutar hasta que no se produzca cierto suceso, como una operación de Entrada/Salida.

Bloqueado - Bloqueado suspendido: Si no hay procesos listos, un proceso bloqueado se pasa a disco para hacer espacio en memoria o para liberar memoria.

Bloqueado suspendido - Bloqueado: Un proceso termina liberando memoria, uno de los bloqueados suspendidos tiene la mayor prioridad y el SO sospecha que el evento que espera sucederá en breve

Bloqueado suspendido - Listo suspendido: Sucede lo que estaba esperando.

Diagrama de 5 estados



Nuevo: Se crea un nuevo proceso para ejecutar un programa.

Nuevo - Listo: Esta transición tiene lugar cuando el Sistema Operativo está preparado para aceptar o admitir un proceso más. Se tendrá en cuenta las restricciones derivadas de la capacidad de la memoria y que no haya tantos procesos activos como para degradar el rendimiento.

Listo - Ejecución: Esta transición se produce cuando el Sistema Operativo selecciona un nuevo proceso que esta en ejecutar en función de su política de planificación.

Ejecución - Listo: La razón más común para esta transición es que el proceso que está en ejecución ha alcanzado el tiempo máximo permitido de ejecución ininterrumpida. Hay otras causas alternativas que no están implementadas en todos los Sistemas Operativos como la expropiación de un proceso en favor de otro más prioritario. Otra situación extraordinaria, que origina esta transición es que un proceso ceda voluntariamente el control del procesador.

Ejecución - Bloqueo: Un proceso realiza esta transición cuando queda a la espera por la concesión de un determinado recurso o por la ocurrencia de un determinado suceso.

Bloqueado - Listo: Tiene lugar si a un proceso bloqueado se le concede el recurso solicitado u ocurre el suceso por el que estaba esperando.

Listo - Terminado: Puede ocurrir si, por ejemplo, un proceso padre decide en un momento determinado finalizar la ejecución de sus procesos hijos. Si alguno de dichos procesos se encontraba en estado preparado realizara esta transición. Otra razón puede ser debido a un requisito de memoria que es denegado.

Bloqueado - Terminado: Un proceso hijo puede realizar esta transición por la misma razón que la anterior. Otra causa puede ser que le proceso supere el tiempo máximo de espera por un recurso y el sistema operativo decida entonces terminarlo (es la razón más habitual).

Cambio de contexto

Subproceso o hilo:

- La CPU guarda el estado actual del hilo (en el TCB) y cambia a otro hilo del mismo proceso.
- Permite a la CPU procesar varios hilos.
- No implica cambio del espacio de memoria, ya que todos los hilos comparten el mismo espacio de proceso al que pertenecen.
- La memoria cache del procesador y TLB conservan su estado.
- Implica el cambio de registro y punteros de pila. Es más eficiente, pero aún así podría llevar varios ciclos de reloj.
- Es un poco más rápido y necesita menos recursos.

Pasos:

1. Se realizan los procesos necesarios para la rutina que almacena los datos necesarios de los registros y estado del thread anterior en el TCB que estará en la estructura de control dentro de la pila del kernel del sistema operativo, almacenado en la memoria RAM.
2. Realización de un proceso de limpieza de la TLB (Translation Lookaside Buffer) cuando sea necesario. Y también se limpiarán otros registros.
3. Se selecciona el siguiente hilo que se ejecutará. La elección puede basarse en diversas políticas de planificación, como prioridad, tiempo de espera, algoritmos de colas, entre otros.
4. Identificación de situaciones en las que se realiza el cambio entre dos hilos dentro del mismo procesador o entre hilos que pertenecen a dos procesos distintos.
5. Procesamiento del nuevo hilo, lo que implica llenar la pipeline con las nuevas instrucciones del hilo actual.
6. Una vez termina y se quiere volver al hilo que se estaba ejecutando anteriormente, el sistema operativo recupera el contexto del hilo seleccionado previamente. Esto implica restaurar los registros del procesador, como el contador de programa y otros datos necesarios para que el hilo pueda continuar su ejecución.
7. Finalmente, se transfiere el control al hilo seleccionado, permitiéndole continuar su ejecución desde el punto donde fue suspendido.

Proceso:

- El planificador del sistema cambia el proceso actual por otro de mayor prioridad y guarda en el PCB el anterior proceso.
- Permite cargar nuevos procesos o programas según la prioridad.
- Implica cambio de espacio de memoria, e incluso podría necesitarse descargar un proceso a la memoria virtual si no hay espacio en la RAM.
- La memoria caché del procesador y el TLB se vacían.
- Es menos eficiente, ya que implica grandes cambios, y puede llevar mayor número de ciclos de reloj, cientos o miles, según la CPU.
- Es relativamente más lento y costoso en recursos.

Pasos:

1. El proceso actual en ejecución en la CPU llega a un punto en el que necesita ser interrumpido o se produce un evento que requiere un cambio de contexto, como una llamada al sistema o una interrupción de hardware.
2. El núcleo del sistema operativo guarda el estado del proceso actual, que incluye los registros de la CPU, el puntero de pila y el contador de programa, en su Bloque de Control de Proceso (PCB). Esto se hace para preservar el estado del proceso y permitir su reanudación en el futuro.
3. El núcleo selecciona el siguiente proceso que debe ejecutarse, ya sea de la cola de procesos listos o siguiendo algún algoritmo de planificación. Luego, actualiza el PCB del nuevo proceso seleccionado y lo pone en estado de «en ejecución».
4. Se llevan a cabo las tareas necesarias para configurar el entorno del nuevo proceso, como cargar los registros de la CPU y el puntero de pila con los valores almacenados en su PCB.
5. El núcleo actualiza las estructuras de memoria, como las tablas de segmentación y las tablas de páginas, para el nuevo proceso.
6. Finalmente, se restaura el contexto del nuevo proceso y se le asigna la CPU para que comience su ejecución. El proceso continúa su ejecución desde el punto en el que fue interrumpido anteriormente.

Interrupciones

Es una suspensión temporal de la ejecución de un proceso, para pasar a ejecutar una subrutina de servicio de interrupción, la cual, por lo general, no forma parte del programa, sino que pertenece al sistema operativo o al BIOS. Surgen de la necesidad de detener a los procesadores en medio de una secuencia de instrucciones para atender otra cosa en ese momento

Tipos de interrupciones

Hardware

- ***Descripción:*** Estas son asíncronas a la ejecución del procesador. Se producen en cualquier momento independientemente de lo que esté haciendo el CPU.
- ***Origen:*** No dependen del procesador, mayormente están ligadas a los dispositivos de E/S.
- ***Tratamiento:*** Esperar hasta recibir los datos o se haya efectuado la transmisión (polling), esta solución bloquea todos los programas en ejecución, no puede admitirse en un sistema multitarea.

Excepciones

- ***Descripción:*** Se producen de forma síncrona a la ejecución del procesador, pueden predecirse si se analiza detenidamente la traza del programa el cual se ejecuta en ese momento.
- ***Origen:*** son causadas al realizarse operaciones no permitidas como son: la división entre 0, el desbordamiento, el acceso a una posición de memoria no permitida, entre otros.
- ***Tratamiento:*** Notifica la condición de error a la aplicación/usuario y abortar la misma.

Software

- ***Descripción:*** generadas por un programa en ejecución.
- ***Origen:*** Se pueden generar instrucciones en el código que permita al desarrollador producir una interrupción.
- ***Tratamiento:*** Deben verificarse bien las líneas de código de los software y ver las limitaciones de los lenguajes con los que fueron creados.

Hilos

En general hay 2 tipos de paralelismo:

- ***Datos (A nivel de Hardware):*** Se centra en distribuir subconjunto de los mismos en múltiples cores y realizar la misma operación en cada core.
- ***Tareas (A nivel de Software):*** Distribuye tareas (Hilos) a través de múltiples cores. Cada hilo realiza una operación única.

Definición

Es una unidad básica de utilización de la CPU, y consiste en un contador de programa, un juego de registros y un espacio de pila. Los hilos dentro de una misma aplicación comparten:

- La sección de código
- La sección de datos
- Los recursos del SO (archivos abiertos y señales)

Los principales estados de un hilo son:

- **Creación:** En general, cuando se crea un nuevo proceso se crea también un hilo para ese proceso. Posteriormente, ese hilo puede crear nuevos hilos dándoles un puntero de instrucción y algunos argumentos. Ese hilo se colocará en la cola de preparados.
- **Bloqueo:** Cuando un hilo debe esperar por un suceso, se le bloquea guardando sus registros. Así el procesador pasará a ejecutar otro hilo preparado.
- **Desbloqueo:** Cuando se produce el suceso por el que un hilo se bloqueó pasa a la cola de listos.
- **Terminación:** Cuando un hilo finaliza, se liberan su contexto y sus pilas.

Características de los hilos:

- La creación, la finalización y el cambio de contexto entre hilos son más rápidos y requieren menos recursos que los procesos.
- Los hilos pueden ejecutarse concurrentemente, lo que significa que múltiples hilos pueden estar en ejecución al mismo tiempo dentro de un proceso. Esto permite aprovechar mejor los sistemas multiprocesador o multinúcleo, puesto que, los hilos pueden distribuirse en los diferentes núcleos de procesamiento.
- **Representación de los Hilos**

Hilos a nivel de usuario:

- La gestión de hilos la realiza una aplicación y el núcleo no es consciente de la existencia de hilos. Se programa una aplicación como multihilo mediante una biblioteca de funciones para gestionar ULT (crear, destruir, comunicar, planificar, etc.)

Hilos a nivel de núcleo (KLT):

- Todo el trabajo de gestión de hilos lo realiza el núcleo. En el área de la aplicación no hay código para la gestión de hilos, únicamente una Interfaz de Programas de Aplicación (API) para las funciones de gestión de hilos en el núcleo.
- Este tipo de soluciones se han implementado en Windows 2000 y Linux. Las aplicaciones se programan como multihilo y todos los hilos de esa aplicación pertenecen al mismo proceso. El núcleo del SO mantiene la información de contexto del proceso como un todo y la de cada hilo dentro del proceso.
- La principal desventaja de la solución KLT, respecto de la ULT, es que el paso de control de un hilo a otro, dentro del mismo proceso, requiere cambios de modo.

- ***Planificación de los Hilos***

Se centra en determinar el orden en el que los hilos accederán al procesador, optimizando la ejecución paralela y minimizando conflictos en el acceso a recursos compartidos.

Criterios de planificación de hilos

Los criterios utilizados en la planificación de hilos comparten similitudes con los de procesos, pero se adaptan a las características particulares de la concurrencia de hilos. Algunos de estos criterios incluyen:

- ***Prioridad de hilos:*** La asignación de prioridades a los hilos es fundamental. Los hilos con mayor prioridad se ejecutan antes, permitiendo que los procesos críticos sean atendidos con prontitud.
- ***Quantum de tiempo:*** La asignación de un quantum de tiempo a cada hilo garantiza una ejecución equitativa. Una gestión adecuada de los quanta contribuye a evitar la monopolización de recursos por parte de un hilo y fomenta la equidad en la ejecución.
- ***Política de prioridad dinámica:*** Similar a la planificación de procesos, la prioridad dinámica permite ajustar las prioridades de los hilos en función de su comportamiento y estado actual. Esta flexibilidad contribuye a una adaptación dinámica a las demandas del sistema.

Planificadores

Definición:

Se refiere al conjunto de políticas y mecanismos incorporados al sistema operativo, a través de un módulo denominado planificador, que debe decidir cuál de los procesos en condiciones de ser ejecutado conviene ser despachado primero y qué orden de ejecución debe seguirse.

Criterios

Criterios Orientados al Sistema

Se centran en el uso efectivo y eficiente del procesador

- ***Productividad:*** La política de planificación debe intentar maximizar el número de procesos terminados por unidad de tiempo. Esta medida indica la cantidad de trabajo que se está realizando. Depende de la longitud media de cada proceso, pero también esta influida por la política de planificación, que puede influir en el uso del procesador.

- **Utilización del procesador:** Es el porcentaje de tiempo en el que el procesador esta ocupado. En un sistema compartido caro, este es un criterio importante. En sistemas monousuarios y en otros sistemas, como los de tiempo real, este criterio tiene menor importancia que otros.

Criterios Orientados al Sistema

- **Equidad:** En ausencia de directrices de usuario o de otras directrices ofrecidas por el sistema, los procesos deben ser tratados de igual forma y ningún proceso debe sufrir inanición.

- **Equilibrio de recursos:** La política de planificación debe mantener ocupados los recursos del sistema. Se debe favorecer a los procesos que no utilicen recursos sobrecargados. Este criterio también afecta a la planificación a medio y largo plazo.

Niveles de planificación

- **Largo:** Realiza el control de admisión de procesos a ejecutar, es muy usada en sistemas batch.
- **Mediano:** Selecciona qué procesos se añaden o se retiran (expulsión a swap) de memoria principal.
- **Corto Plazo:** Selecciona el siguiente proceso a ejecutar.

Expropiativos: Permiten que un proceso con mayor prioridad quite la CPU al proceso que está ejecutando antes de su finalización.

No expropiativos: una vez que un proceso obtiene la CPU no se le puede quitar hasta que no acabe su ejecución.

Políticas: Las políticas de planificación determinan qué se hará

- Apropiativa o no
- Igualitaria o no
- Eficiente o no
- Predecible o no

Mecanismos: Determinan cómo se hará, implementado a partir de un conjunto de algoritmos:

- FCFS
- SJF,
- SRTF
- RR
- COLAS MULTINIVEL

Por Prioridad: consiste en asignar un valor cuantificable que le da más prioridad a unos procesos que a otros. Hay dos tipos de prioridades:

- **Estáticas:** se mantiene la prioridad asignada durante la ejecución.
- **Dinámicas:** se varía la prioridad durante la ejecución.

Algoritmos de Planificación

Para todos los algoritmos vistos en clase: Cómo trabajan, requerimientos del sistema para administrarlos, Ventajas y Desventajas.

FCFS (First-Come, First-Served)

Este algoritmo es muy sencillo como simple, pero también el de menor rendimiento, en este mismo, el primer proceso que llega, se ejecuta y una vez terminado va por el siguiente.

Sus características son:

- No apropiativa.
- Es justo, aunque los procesos largos hacen esperar mucho a los cortos.
- Predecible.
- El tiempo medio de servicio es muy variable en función del número de procesos y su duración

SJF (Shortest Job First)

Prioriza los procesos más cortos independientemente de su llegada y en caso de que los procesos utilicen la misma cantidad de tiempo, se implementa el método anterior. Este sistema tiene el riesgo de poner siempre al final de la cola los procesos más largos por lo que nunca se ejecutarán, esto se conoce como inanición.

Ventaja: Este algoritmo asegura que el procesador siempre esté ejecutando el proceso más corto disponible, lo que lleva a una alta utilización del procesador y una respuesta rápida del sistema.

Desventaja: Es difícil de implementar en un sistema real, ya que requiere información precisa sobre la duración de los procesos.

SRTF (Short Remaining Time Next)

Añadiendo la expulsión de procesos al algoritmo SJF obtenemos SRTF, éste será capaz de expulsar un proceso largo en ejecución para ejecutar otros más cortos. El problema que puede surgir es que un proceso largo puede llegar a expulsarse muchas veces y nunca terminar debido a la ejecución de otros más cortos.

Se ha convertido en una herramienta clave para la optimización de tiempo de ejecución. Esto se debe a su capacidad para minimizar el tiempo total de ejecución de los procesos, en comparación con otros algoritmos de planificación de procesos.

Round Robin

Asigna a cada proceso un tiempo equitativo tratando a todos los procesos por igual y con la misma prioridad.

Este algoritmo es circular, volviendo siempre al primer proceso una vez terminado con el último, para controlar este método a cada proceso se le asigna un intervalo de tiempo llamado quantum o cuanto. Pueden suceder dos casos con este método:

El proceso es menor que el quantum: Al terminar antes se planifica un nuevo proceso.

El proceso es mayor que el quantum: Al terminar el quantum se expulsa el proceso dando paso al siguiente proceso en la lista. Al terminar la iteración se volverá para terminar el primer proceso expulsado

Ventaja: Asegura una distribución equitativa del tiempo de procesamiento. Además, como cada proceso recibe un tiempo de procesamiento limitado, ningún proceso puede bloquear el procesador indefinidamente.

Desventaja: Puede llevar a una latencia adicional debido al tiempo de cambio de contexto. Es decir, el tiempo que se tarda en cambiar de un proceso a otro puede ralentizar el sistema.

Prioridades

A cada proceso se le asocia una prioridad y a la CPU se asigna el proceso que tenga la prioridad más alta. Sin embargo, los procesos con la misma prioridad se planifican en orden FCFS. Generalmente, las prioridades se indican mediante un rango de números fijo, como por ejemplo de 0 a 7, o de 0 a 4095.

Se pueden definir en interna o externa. Las internas utilizan algún valor mensurable para calcular la prioridad de un proceso. Las externas se establece en función de criterios externos al sistema operativo.

¿Cuándo utilizar el algoritmo de planificación de prioridades?

Sistemas en tiempo real: Cuando existen procesos críticos que requieren una respuesta inmediata, como en sistemas de control industrial, aeronáuticos o médicos.

Sistemas interactivos: En sistemas donde la respuesta rápida es esencial, como en interfaces gráficas de usuario o editores de texto.

Sistemas batch: En entornos donde se ejecutan tareas independientes, como en sistemas de procesamiento de datos por lotes.

Planificación mediante colas multinivel

Divide la cola de procesos preparados en varias colas distintas. Los procesos se asignan permanentemente a una cola, generalmente en función de alguna propiedad del proceso. Cada cola tiene su propio algoritmo de planificación. Las prioridades pueden ser las siguientes:

1. Procesos del sistema
2. Procesos interactivos.
3. Procesos de edición interactivos.
4. Procesos por lotes.
5. Procesos de estudiantes

Apropiativa: Es decir si llega un proceso con mayor prioridad que el que se está ejecutando podrá expulsarlo y apropiarse del procesador.

Cada cola tiene su prioridad: Cuando un proceso entra en la cola, automáticamente se calculará su prioridad interna.

Planificación mediante colas multinivel realimentadas

Permite mover un proceso de una cola a otra. La idea es separar los procesos en función de las características de sus ráfagas de CPU. Si un proceso utiliza demasiado tiempo de CPU, se pasa a una cola de prioridad más baja. Además, un proceso que esté esperando demasiado tiempo en una cola de baja prioridad puede pasarse a una cola de prioridad más alta.

Características

- ***Priorización dinámica:*** Se ajustan dinámicamente en función de su comportamiento y uso de recursos.
- ***Ejecución basada en comportamiento:*** Los procesos que utilizan más recursos pueden descender en la jerarquía de colas para dar prioridad a otros procesos.

- **Retroalimentación:** Los procesos reciben retroalimentación del sistema en forma de ajustes en su prioridad en función de su comportamiento.

Ventajas

Procesos limitados por E/S: Obtienen prioridad alta y se les asigna la CPU rápidamente. Abandonan la red de colas cuando solicitan E/S, liberando recursos para otros procesos.

Procesos limitados por CPU: Reciben un cuanto mayor en las colas inferiores. Esperan más tiempo entre ejecuciones, pero utilizan todo su quantum.

Sistemas de Tiempo Real

Es un sistema informático que interacciona con su entorno físico y responde a los estímulos del entorno dentro de un plazo de tiempo determinado.

Existen sistemas de tiempo real crítico (tiempo real duro), en donde los plazos de respuesta deben respetarse estrictamente y una sola respuesta tardía a un suceso externo puede tener consecuencias fatales, sistemas de tiempo real acrítico (tiempo real suave), en los que se pueden tolerar retrasos ocasionales en la respuesta a un suceso.

La planificación estática dirigida por tabla

Utilizado para gestionar tareas periódicas. Se basa en la creación de un plan predefinido que determina cuándo se ejecutará cada tarea. Este enfoque ofrece un alto grado de predictibilidad, pero puede ser inflexible ante cambios en las necesidades del sistema.

La planificación estática con expulsión dirigida por prioridad

Se basa en dos conceptos clave:

Asignación de prioridades estáticas: Cada tarea se asocia con una prioridad fija que define su importancia en el sistema. La prioridad se asigna en función de las restricciones de tiempo de la tarea, como su plazo y tiempo de ejecución.

Planificación expulsiva: Cuando una nueva tarea llega al sistema, se compara su prioridad con la de las tareas en ejecución. Si la prioridad de la nueva tarea es mayor, se expulsa una de las tareas en ejecución con menor prioridad para liberar recursos. La tarea expulsada se reanudará en un momento posterior, cuando haya recursos disponibles.

Planificación dinámica basada en un plan

Se basa en la creación de un plan flexible que se adapta a la llegada de nuevas tareas. Este enfoque ofrece un equilibrio entre eficiencia y flexibilidad, esto lo hace adecuado para sistemas de tiempo real con requisitos dinámicos.

La planificación dinámica de mejor esfuerzo

Ampliamente utilizado en los sistemas operativos de tiempo real disponibles comercialmente. Se basa en los siguientes principios:

Asignación de prioridades

Cuando llega una nueva tarea al sistema, se le asigna una prioridad en función de sus características. Las características que se consideran para la asignación de prioridades pueden incluir:

Plazo: Tiempo límite para la finalización de la tarea.

Criticidad: La importancia de la tarea para el funcionamiento del sistema.

Recursos necesarios: La cantidad de recursos que la tarea necesita para ejecutarse.

Planificación basada en plazos

Se utiliza un algoritmo de planificación basado en plazos, como la planificación del plazo más cercano (EDF), para ordenar las tareas en la cola de listos. El algoritmo EDF prioriza las tareas con plazos más tempranos.

Naturaleza no periódica de las tareas

Las tareas en este tipo de planificación no son periódicas, lo que significa que no tienen un patrón de llegada y ejecución predecible. Esto implica que no es posible realizar un análisis estático de planificabilidad para determinar si todas las tareas podrán cumplir con sus plazos antes de su ejecución.

Incertidumbre en el cumplimiento de plazos

Con la planificación dinámica de mejor esfuerzo, no se puede garantizar el cumplimiento de las restricciones de tiempo de las tareas hasta que su plazo venza o la tarea se complete. Esta incertidumbre es la principal desventaja de este enfoque.

Facilidad de implementación

La principal ventaja de la planificación dinámica de mejor esfuerzo es su facilidad de implementación. No requiere un conocimiento a priori del comportamiento de las tareas, lo que la hace adaptable a una amplia gama de sistemas de tiempo real.

Planificación para Linux

Es un algoritmo basado en prioridades y apropiativo, con dos rangos distintos de prioridades: un rango de tiempo real de 0 a 99 y un valor normal (nice) en el rango comprendido entre 100 y 140. Estos dos rangos se asignan a un esquema de prioridades global, en el que los valores numéricamente más bajos indican las prioridades más altas.

Linux asigna a las tareas de prioridad más alta, a los tiempos más largos y a las tareas de prioridad más baja cuantos de tiempo más cortos.

Planificación para Windows

Utiliza un sistema de multi-cola ordenadas por prioridades. Son 32 niveles de prioridades (de 0 a 31) que se dividen de la siguiente forma:

Niveles 16 a 31: Tiempo Real

Niveles 1 a 15: Variable

Nivel 0: Reservado para el sistema

El planificador utiliza Round Robin para recorrer las distintas colas. Tiene un quantum fijo que por defecto valdrá $q=2$ o $q=12$, dependiendo de si es un SO de servidor o de usuario.

Planificación en Multiprocesadores

Los procesadores pueden ser de dos tipos: idénticos o diferentes. Si son diferentes, las opciones son limitadas: cada procesador tiene su propia cola y su propio algoritmo de planificación.

Con varios procesadores idénticos, se tiene una carga compartida: colas separadas para cada procesador. Pero un procesador podría estar sobresaturado, mientras otro permanece reposo. Para ello, se utiliza una cola común. Así, todos los trabajos están en una cola y planifican sobre cualquier procesador disponible.

Se pueden dar dos enfoques de planificación en multiprocesadores:

Cada procesador se autoplanifica: Cada procesador analiza la cola y escoge un proceso para ejecutar, pero se debe asegurar que dos procesadores no escogieron el mismo proceso para ejecutar así como también no perder procesos en la cola.

Un procesador planifica a los demás: Se tiene una estructura Rey/súbditos. El procesador principal ejecuta el sistema operativo y las rutinas principales, los súbditos solo ejecutan programas de usuarios. El problema surge si se queda bloqueado el Rey.

Planificación en Sistemas Multicore

Es considerado un problema NP (Polinomial No Determinista) y no es posible obtener la solución óptima. Los algoritmos heurísticos de planificación de tareas son considerados los más adecuados para encontrar las soluciones subóptimas para problemas NP. Comúnmente se pueden clasificar en tres tipos: algoritmo de planificación de tareas basado en listas, algoritmo basado en duplicación de tareas y basado en clústeres.

Algoritmo de planificación de tareas basado en listas

Lista con prioridades estáticas (LSSP): Donde las tareas se programan en el procesador que puede iniciarlas más temprano según el orden de sus prioridades, que se calculan de antemano.

Programación en lista con prioridades dinámicas (LSDP): para este último, las prioridades de las tareas no programadas deben recalcularse después de finalizar un proceso

algoritmo basado en duplicación de tareas

Para disminuir el tiempo de comunicación entre tareas en diferentes núcleos, algunas tareas pueden duplicarse en los núcleos a los que se asignan sus tareas sucesoras después de haber sido asignadas a diferentes núcleos, lo que puede reducir aún más la longitud total de la programación.

Los algoritmos basados en clústeres

Tiene 2 etapas:

Mapeo: Las tareas se asignan a diferentes clústeres de acuerdo con la política especificada.

Programación: Las tareas del mismo clúster se programan en el mismo núcleo.

Planificación de hilos

Son unidades de ejecución independientes dentro de un proceso. Permiten que un solo proceso ejecute múltiples tareas de forma simultánea, lo que mejora la eficiencia y el rendimiento. Existen dos tipos principales de hilos:

Hilos a nivel de usuario: Son administrados por una biblioteca de hilos en el espacio de usuario del sistema operativo. El kernel no tiene conocimiento directo de estos hilos.

Hilos a nivel de kernel: Son creados y administrados directamente por el kernel del sistema operativo.

Tipos de planificación por hilos

Compartición de carga: Los procesos no se asignan a un procesador particular. Se mantiene una cola global de hilos listos, y cada procesador, cuando está ocioso, selecciona un hilo de la cola.

Planificación en pandilla: Un conjunto de hilos relacionados que se planifica para ejecutar sobre un conjunto de procesadores al mismo tiempo, en una relación uno-a-uno.

Asignación de procesador dedicado: Proporciona una planificación implícita definida por la asignación de hilos a procesadores. Cada proceso ocupa un número de procesadores igual al número de hilos en el programa, durante toda la ejecución del programa. Cuando el programa termina, los procesadores regresan al estado de ocio, esperando la posible asignación a otro programa.

Planificación dinámica: El número de hilos de un proceso puede cambiar durante el curso de su ejecución.

Tema II-B Sincronización y Comunicación entre Procesos

Procesos independientes: Independientes funcionan independientemente de sí mismos. No afectan ni pueden verse afectados por otros procesos que se ejecutan dentro del sistema operativo, ni comparten datos con ningún proceso o sistema.

Procesos cooperantes: Pueden verse afectados por otros procesos y, a su vez, afectar a otros procesos dentro del sistema operativo. Estos procesos comparten sus datos con otros procesos y sistemas.

Condición de carrera o competencia: Situación en la que dos o mas procesos intentan realizar alguna operación (Lectura-Escritura) en ciertos datos compartidos (In, Out) y el resultado final depende de quién ejecuta qué y en qué momento

Exclusión Mutua:

Cuando un proceso ejecuta código que manipula datos compartidos (o recursos), se dice que el proceso está en su Sección Crítica (SC) (para esos datos compartidos).

4 condiciones para una herramienta que garantice la EM

1. ***Exclusión mutua:*** No puede haber dos procesos simultáneamente en la región crítica. Si un proceso P_i se está ejecutando en su región crítica, ningún otro proceso se puede ejecutar en dicha región crítica.

2. ***Progreso:*** Ningún proceso fuera de la región crítica puede bloquear a otros procesos.

3. ***Espera acotada:*** Ningún proceso debe esperar infinitamente para entrar en su región crítica.

4. ***Los procesos se ejecutan a una velocidad >0***

- Su velocidad relativa no influye
- No se realiza ninguna asunción sobre la velocidad de los procesos o sobre el número de CPUs

Sincronización

Es un conjunto de protocolos y mecanismos utilizados para preservar la integridad y consistencia del sistema, cuando varios procesos concurrentes comparten recursos que son reutilizables en serie (sólo pueden ser utilizados por un proceso a la vez). Su estado y operación pueden resultar corrompidos si son manipulados concurrentemente y sin sincronización por más de un proceso. Ej: variables compartidas para Lectura Escritura, dispositivos físicos como impresoras, entre otros.

Para cada herramienta estudiada: Clasificar Sincronización, señalización, comunicación; Protocolo de Negociación Liberación; ventajas y desventajas.

Soluciones de SW

Algoritmo de Dekker

Fue la primera solución “correcta” al problema de la sección crítica. Permite que dos subprocesos compartan un recurso sin conflicto, utilizando la memoria compartida para la comunicación. Evita la alternancia estricta de un algoritmo de toma de turnos ingenuo, por otra parte fue uno de los primeros algoritmos de exclusión mutua que se inventaron.

Versión 1

Principalmente utiliza un número de hilos común o compartido entre procesos, para así evitar que el otro proceso ingrese a su sección crítica, si el hilo compartido indica que el anterior ya está en ejecución. El problema que surge en la implementación anterior es la sincronización en lockstep, es decir, cada hilo depende del otro para su ejecución. Si uno de los procesos se completa, entonces el segundo proceso se ejecuta, le da acceso al completado y espera su turno; sin embargo, el primer proceso ya está completado y nunca se ejecutaría para devolver el acceso al segundo. Por lo tanto, el segundo proceso espera indefinidamente

```

Main()
{
    int thread_number = 1;
    startThreads();
}

Thread1()
{
    do {

        // entry section
        // wait until threadnumber is 1
        while (threadnumber == 2)
            ;

        // critical section

        // exit section
        // give access to the other thread
        threadnumber = 2;

        // remainder section

    } while (completed == false)
}

Thread2()
{
    do {

        // entry section
        // wait until threadnumber is 2
        while (threadnumber == 1)
            ;

        // critical section

        // exit section
        // give access to the other thread
        threadnumber = 1;

        // remainder section

    } while (completed == false)
}

```

Versión 2

Para eliminar la sincronización en lockstep, utiliza dos banderas para indicar su estado actual y las actualiza de acuerdo con la sección de entrada y salida. Si los hilos son forzados a interrumpirse (detenidos) durante la actualización de la bandera (es decir, durante `current_thread = true`), entonces ambos hilos entran en su sección crítica una vez que el hilo forzado a interrumpirse se reinicia. Lo mismo se puede observar al principio, cuando ambas banderas son falsas.


```

Main()
{
    // flags to indicate if each thread is in
    // its critical section or not.
    boolean thread1 = false;
    boolean thread2 = false;

    startThreads();
}

Thread1()
{
    do {
        // entry section
        // wait until thread2 is in its critical section
        while (thread2 == true)
            ;

        // indicate thread1 entering its critical section
        thread1 = true;

        // critical section

        // exit section
        // indicate thread1 exiting its critical section
        thread1 = false;

        // remainder section

    } while (completed == false)
}

Thread2()
{
    do {
        // entry section
        // wait until thread1 is in its critical section
        while (thread1 == true)
            ;

        // indicate thread2 entering its critical section
        thread2 = true;

        // critical section

        // exit section
        // indicate thread2 exiting its critical section
        thread2 = false;

        // remainder section

    } while (completed == false)
}

```

Versión 3

Para volver a garantizar la exclusión mutua, establece las banderas antes de la propia sección de entrada. El problema con esta versión es la posibilidad de un bloqueo. Ambos hilos podrían establecer su bandera como verdadera simultáneamente y ambos esperarían infinitamente más tarde

```

Main()
{
    // flags to indicate if each thread is in
    // queue to enter its critical section
    boolean thread1wantstoenter = false;
    boolean thread2wantstoenter = false;

    startThreads();
}

Thread1()
{
    do {
        thread1wantstoenter = true;

        // entry section
        // wait until thread2 wants to enter
        // its critical section
        while (thread2wantstoenter == true)
            ;

        // critical section

        // exit section
        // indicate thread1 has completed
        // its critical section
        thread1wantstoenter = false;

        // remainder section
    } while (completed == false)
}

Thread2()
{
    do {
        thread2wantstoenter = true;

        // entry section
        // wait until thread1 wants to enter
        // its critical section
        while (thread1wantstoenter == true)
            ;

        // critical section

        // exit section
        // indicate thread2 has completed
        // its critical section
        thread2wantstoenter = false;

        // remainder section
    } while (completed == false)
}

```

Versión 4

Utiliza un pequeño intervalo de tiempo para volver a verificar la condición, elimina el bloqueo y también garantiza la exclusión mutua. El problema con esta versión es el aplazamiento indefinido. Además, una cantidad de tiempo aleatoria es errática dependiendo de la situación en la que se está implementando el algoritmo, por lo tanto, no es una solución aceptable en sistemas críticos para los negocios.

```

Main()
{
    // flags to indicate if each thread is in
    // queue to enter its critical section
    boolean thread1wantstoenter = false;
    boolean thread2wantstoenter = false;

    startThreads();
}

Thread1()
{
    do {
        thread1wantstoenter = true;

        while (thread2wantstoenter == true) {

            // gives access to other thread
            // wait for random amount of time
            thread1wantstoenter = false;

            thread1wantstoenter = true;
        }

        // entry section
        // wait until thread2 wants to enter
        // its critical section

        // critical section

        // exit section
        // indicate thread1 has completed
        // its critical section
        thread1wantstoenter = false;

        // remainder section
    } while (completed == false)
}

Thread2()
{
    do {
        thread2wantstoenter = true;

        while (thread1wantstoenter == true) {

            // gives access to other thread
            // wait for random amount of time
            thread2wantstoenter = false;

            thread2wantstoenter = true;
        }

        // entry section
        // wait until thread1 wants to enter
        // its critical section

        // critical section

        // exit section
        // indicate thread2 has completed
        // its critical section
        thread2wantstoenter = false;

        // remainder section
    } while (completed == false)
}

```

Versión 5

La idea es utilizar la noción de hilo favorecido para determinar la entrada a la sección crítica. El hilo favorecido alterna entre el hilo que proporciona la exclusión mutua y evita el bloqueo, el aplazamiento indefinido o la sincronización en lockstep. Esta versión garantiza una solución completa al problema de la sección crítica.

```

Main()
{
    // to denote which thread will enter next
    int favouredthread = 1;

    // flags to indicate if each thread is in
    // queue to enter its critical section
    boolean thread1wantstoenter = false;
    boolean thread2wantstoenter = false;

    startThreads();
}

Thread1()
{
    do {

        thread1wantstoenter = true;

        // entry section
        // wait until thread2 wants to enter
        // its critical section
        while (thread2wantstoenter == true) {

            // if 2nd thread is more favored
            if (favouredthread == 2) {

                // gives access to other thread
                thread1wantstoenter = false;

                // wait until this thread is favored
                while (favouredthread == 2)
                    ;

                thread1wantstoenter = true;
            }
        }

        // critical section

        // favor the 2nd thread
        favouredthread = 2;

        // exit section
        // indicate thread1 has completed
        // its critical section
        thread1wantstoenter = false;

        // remainder section

    } while (completed == false)
}

```

```

Thread2()
{
    do {
        thread2wantstoenter = true;

        // entry section
        // wait until thread1 wants to enter
        // its critical section
        while (thread1wantstoenter == true) {

            // if 1st thread is more favored
            if (favaouredthread == 1) {

                // gives access to other thread
                thread2wantstoenter = false;

                // wait until this thread is favored
                while (favouredthread == 1)
                    ;

                thread2wantstoenter = true;
            }
        }

        // critical section

        // favour the 1st thread
        favouredthread = 1;

        // exit section
        // indicate thread2 has completed
        // its critical section
        thread2wantstoenter = false;

        // remainder section
    } while (completed == false)
}

```

Algoritmo de Peterson

Es una solución de software al problema del acceso concurrente a un recurso compartido, como un archivo o una base de datos. Esto funciona haciendo que los procesos, soliciten permiso al controlador llamado “coordinador”. Asimismo, el coordinador concede el permiso a un proceso si necesita acceder a la “sección crítica”, mientras uno accede a la sección crítica, los demás están “en espera”. Por otra parte, los procesos son atendidos por el coordinador en orden de llegada.

```

bandera[0] = false
bandera[1] = false
turno      // No es necesario asignar un turno
p0: bandera[0] = true          p1: bandera[1] = true
    turno = 1                  turno = 0
    while( bandera[1] && turno == 1 );    while( bandera[0] && turno == 0 );
        { //no hace nada; espera. }      { //no hace nada; espera. }
    // sección crítica                // sección crítica

    // fin de la sección crítica        // fin de la sección crítica
    bandera[0] = false                bandera[1] = false

```

Para 2 procesos

Los procesos P0 y P1 no pueden estar en la sección crítica al mismo tiempo. Si P0 está en la sección crítica, entonces bandera[0] = true, y ocurre que bandera[1] = false, con lo que P1 ha terminado la sección crítica, o que la variable compartida turno = 0, con lo que P1 está esperando para entrar a la sección crítica. En ambos casos, P1 no puede estar en la sección crítica.

Algoritmo de Lamport

Los principios operativos detrás del algoritmo son muy simples. Todos los subprocesos del proceso deben tomar un número y esperar su turno para usar un recurso informático compartido o para ingresar a su sección crítica. El número puede ser cualquiera de las variables globales y los procesos con el número más bajo se procesarán primero. Si hay un empate o un número similar compartido por ambos procesos, se gestiona a través de su ID de proceso. Si un proceso termina antes de su turno, debe comenzar de nuevo en la cola de procesos.

Soluciones Hw:

Habilitar/Deshabilitar Interrupciones

Muchos sistemas proporcionan soporte hardware para secciones críticas

Uniprocesadores

- Se pueden deshabilitar las interrupciones
- El código se ejecuta sin expulsión: poco recomendable
- Generalmente muy ineficiente en multiprocesadores
- Hace que el SO no escale bien

• **Habilitar/Deshabilitar Interrupciones**

Process Pi

repeat

disable interrupts

critical section

enable interrupts

remainder section

forever

Test and Set

Es una instrucción utilizada para escribir (establecer) un 1 en una ubicación de memoria y devolver su valor anterior como una operación atómica (es decir, no interrumpible). El llamador puede entonces "probar" el resultado para verificar si el estado fue modificado por la llamada

```
//Shared variable lock initialized to false
boolean lock;

boolean TestAndSet (boolean &target){
    boolean rv = target;
    target = true;
    return rv;
}

while(1){
    while (TestAndSet(lock));
    critical section
    lock = false;
    remainder section
}
```

Siempre devuelve el valor que se le pasa y establece lock en true. El primer proceso entrará a la sección crítica inmediatamente, porque TestAndSet(lock) devolverá false y saldrá del bucle while. Los demás procesos no pueden entrar, debido a que lock se ha establecido en true, por lo que el bucle while seguirá siendo verdadero. Se garantiza la exclusión mutua.

Una vez que el primer proceso sale de la sección crítica, lock cambia a false. Ahora, los demás procesos pueden entrar uno por uno. También se garantiza el progreso. Sin embargo, después del primer proceso, cualquier proceso puede entrar. No se mantiene una cola, por lo que cualquier nuevo proceso que encuentre el lock en false podrá entrar. Por lo tanto, no se garantiza la espera acotada (bounded waiting).

Ventajas

Atomicidad garantizada: La operación se realiza de manera atómica, evitando condiciones de carrera (race conditions) y garantizando la exclusión mutua entre procesos o hilos que comparten una variable.

Simplicidad: La implementación de Test-and-Set es simple tanto a nivel de hardware como de software. Esto la hace adecuada para soluciones rápidas de sincronización, como spinlocks.

Soporte en hardware: Muchas arquitecturas de procesadores modernos soportan Test-and-Set, lo que la convierte en una opción estándar para manejar la sincronización entre hilos o procesos.

Rendimiento en entornos pequeños: En sistemas con pocos hilos o procesos, Test and-Set puede ofrecer una sincronización eficiente sin necesidad de estructuras adicionales, como colas o semáforos.

Desventajas

Espera activa (Busy-waiting): Cuando un proceso no puede adquirir el bloqueo porque otro ya lo tiene, entra en un bucle de espera activa (spinlock), consumiendo recursos de la CPU innecesariamente.

No garantiza espera acotada (Bounded waiting): No asegura a los procesos que esperan por el bloqueo, sean atendidos en orden o dentro de un tiempo finito. Esto puede llevar a starvation (procesos que esperan indefinidamente).

Problemas de rendimiento en sistemas multiprocesador: En arquitecturas con múltiples procesadores, puede causar congestión en el bus de memoria, afectando el rendimiento global del sistema.

Falta de justicia: Cualquier proceso que intente realizar un Test-and-Set puede adquirir el bloqueo tan pronto como se libere, sin respetar el orden de llegada de los procesos.

Eficiencia decreciente con alta contención: A medida que aumenta el número de procesos que compiten por un recurso, la contención hace que la eficiencia de Test-and-Set disminuya significativamente debido a la constante invalidación de caché entre procesadores.

Compare and Swap

Es una instrucción atómica utilizada en programación con múltiples hilos para lograr la sincronización. Compara el contenido de una ubicación de memoria con un valor dado, solo si son iguales, modifica el contenido de esa ubicación de memoria con un nuevo valor proporcionado. Esto se realiza como una única operación atómica

En lugar de establecer directamente lock en true dentro de la función de intercambio (swap), se establece key en true y luego se intercambia con lock.

El primer proceso se ejecutará, y en el bucle while(key), como key=true, realiza el intercambio, lo que establece lock=true y key=false. En la siguiente iteración de while(key), como key=false, el bucle se rompe y el primer proceso entra en la sección crítica.

Ahora, otro proceso intentará entrar en la sección crítica, por lo que nuevamente `key=true`. El bucle `while(key)` se ejecutará y se realizará el intercambio, estableciendo `lock=true` y `key=false`. En la siguiente iteración, como `while(key)` es verdadero, este proceso continuará ejecutándose y otro proceso no podrá entrar en la sección crítica.

Cuando el proceso sale de la sección crítica, `lock` se cambia a `false`, por lo que cualquier proceso que lo encuentre podrá entrar en la sección crítica.

```
// Shared variable lock initialized to false
// and individual key initialized to false;

boolean lock;
Individual key;

void swap(boolean &a, boolean &b){
    boolean temp = a;
    a = b;
    b = temp;
}

while (1){
    key = true;
    while(key)
        swap(lock, key);
    critical section
    lock = false;
    remainder section
}
```

Ventajas

Atomicidad: La operación es atómica, lo que significa que puede ejecutarse sin interrupciones, garantizando la exclusión mutua en entornos con múltiples hilos o procesadores.

Sin necesidad de bloqueos (lock-free): Permite implementar algoritmos libres de bloqueos, reduciendo problemas como interbloqueos (deadlocks) y esperas activas prolongadas.

Rendimiento en sistemas con baja contención: Este puede ser más eficiente que los enfoques basados en bloqueos tradicionales, ya que evita bloquear y desbloquear explícitamente.

Flexibilidad: Se utilizarse para construir estructuras y algoritmos avanzados, como pilas, colas y contadores, que sean seguros para múltiples hilos sin bloquear.

Compatibilidad con múltiples arquitecturas: Muchas arquitecturas modernas de procesadores ofrecen soporte nativo para CAS, lo que facilita su uso en diferentes sistemas.

Desventajas

Problemas de contención alta: Cuando varios hilos intentan actualizar la misma variable simultáneamente, Este puede fallar repetidamente, lo que provoca que algunos hilos tengan que reintentar muchas veces, reduciendo el rendimiento.

Falta de espera acotada: No garantiza que todos los hilos en espera logren completar su operación en un tiempo finito. Esto puede llevar a problemas de starvation en escenarios de alta competencia.

Complejidad para estructuras más grandes: Opera sobre un único valor en memoria. Si es necesario modificar varios valores de manera atómica, se requiere un diseño más complejo, como la implementación de versiones o registros adicionales.

Mayor consumo de CPU en caso de reintentos: Si el valor objetivo cambia con frecuencia debido a actualizaciones de otros hilos, los reintentos pueden consumir una cantidad considerable de tiempo de CPU, especialmente en sistemas altamente concurridos.

Dificultad para depurar: Son más difíciles de diseñar y depurar debido a su naturaleza libre de bloqueos y la posibilidad de condiciones de carrera sutiles.

Semáforos

Es un mecanismo propuesto por Dijkstra (1965), para la exclusión mutua entre un número arbitrario de procesos

- Un semáforo S es una variable entera, sobre la cual se definen tres (3) operaciones:

1. Inicialización en un valor no negativo

2. Wait (s) Disminuye el valor del semáforo P (Proberen) Probar en holandés

3. Signal (s) Incrementa el valor del semáforo V (Verhogen) Incrementar en holandés

Consta de dos operaciones primitivas: Señal-Signal-V y Espera Wait- P que operan sobre el tipo especial de variable semáforo S:

- Wait (S): Decrementa el valor de su argumento semáforo S, en tanto que el resultado no sea negativo. La concusión de la operación wait, una vez tomada la decisión de decremento a su argumento semáforo, debe ser indivisible

- Signal (S): Incrementa el valor de su argumento semáforo S, en una operación indivisible

Binarios

Pueden tomar valor 0 si el RC está ocupado y 1 si está libre

- Wait (s) es la espera hasta que la variable semáforo sea igual a LIBRE, seguido de su modificación indivisible para que indique OCUPADO antes de devolver el control al invocador. La operación wait implementa por tanto la fase de negociación del protocolo de EM.

- Signal (s) pone el valor de la variable en Libre y representa la fase de liberación

Contadores

- Pueden tomar cualquier valor entero no negativo
- Son particularmente útiles cuando hay que asignar un recurso a partir de un conjunto de recursos idénticos
- Tienen como valor inicial el número de recursos del conjunto
- Cada operación Wait(s) decrementa el semáforo s en 1 indicando que se ha retirado un recurso del grupo y lo está utilizando algún proceso
- Cada operación Signal (s) incrementa en 1 el semáforo lo que indica la devolución de un recurso al grupo y que el recurso puede ser reasignado
- Si se intenta una operación Wait cuando el semáforo tiene valor 0, el proceso deberá esperar hasta que se libere un recurso, mediante una operación Signal

Soluciones del SO:

Implantación de semáforos con Colas

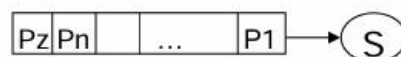
Implantación

Wait(s): Si no ($s > 0$) entonces

Suspender al proceso invocador en S
en caso contrario
 $s := s - 1;$

Signal(s): Si no cola vacía (al menos un proceso espera) entonces

Reanudar uno de los procesos en la cola de s
en caso contrario
 $s := s + 1;$



Estructura del Semáforo

- Las operaciones de semáforos se pueden incluir en el núcleo del SO para evitar la espera activa.
- Un semáforo se implanta con una variable protegida y una cola de PCBs en la cual los procesos esperan por operaciones V
- Cuando un proceso intenta una operación P sobre un semáforo en 0, renuncia al procesador y se bloquea a sí misma para esperar por una operación V sobre el semáforo
- El SO inserta el PCB del proceso en la cola del semáforo asociado al recurso con una política FIFO
- Cuando el recurso quede finalmente disponible, una operación V sobre el semáforo permite reanudar al proceso suspendido (quita el proceso de la cola de bloqueados y lo pasa a la cola de listos)
- La operación V intenta despertar un proceso que espera y solo cuando no hay ninguno incrementa la variable semáforo (la variable semáforo se encuentra en estado ocupado mientras haya procesos esperando en la cola)
- Un proceso nuevo que ejecute P será insertado en la cola siempre que el semáforo esté ocupado

Soluciones Lenguajes de Programación:

Monitores

Definición

Son estructuras de un lenguaje de programación que ofrecen funcionalidad equivalente a los semáforos pero mas fáciles de controlar. Definidos por Hoare (1974), se han implantado en lenguajes como Pascal Concurrente, Modula-2, Modula-3, Pascal Plus.

Es un modulo software que consta de uno o más procedimientos, una secuencia de inicialización y unos datos locales.

Se caracterizan por

- Las variables de datos solo son accesibles a los procedimientos del monitor y no por los procedimientos externos
- Un proceso entra al monitor invocando uno de los procedimientos

- Un solo procedimiento puede estar ejecutándose en el monitor en un instante dado; cualquier otro proceso que haya invocado al monitor quedara suspendido, esperando que el monitor se libere

Mecanismo de Sincronización

Es provisto a través de las variables de tipo condition, con dos operaciones

cwait(c): Suspende la ejecución del proceso llamado bajo la condición c. El monitor queda disponible para ser usado por otro proceso

csignal(c): Reanuda la ejecución de algún proceso suspendido después de un cwait bajo la misma condición. Si hay varios procesos elige uno de ellos y si no hay no se hace nada.

Type monitor-name = monitor

Variable declarations

Procedure entry P1 (...);

begin end;

Procedure entry P2(...);

begin end;

begin

initialization code

end;

Comunicación:

Paso de Mensajes

Definición

- Son un mecanismo relativamente sencillo, adecuado para garantizar tanto la EM como la comunicación entre procesos en entornos centralizados y distribuidos
- El envío y recepción de mensajes es una forma estándar de comunicación entre nodos de una red de computadores
- Un mensaje es una colección de información que puede ser intercambiada entre un proceso emisor y receptor, puede contener datos, ordenes de ejecución e incluso código a transmitir entre dos o mas procesos

- Un mensaje se caracteriza por tipo, longitud, identificadores de emisor y receptor y un campo datos.

Denominación

- **Denominación Directa:** Cuando se invoca a una operación de mensajes, cada emisor debe designar al receptor específico y a la inversa, cada receptor debe designar a la fuente desde la cual se desea recibir un mensaje, por lo cual es una comunicación simétrica
 - **Problema:** Inconveniente para rutinas de servicio, por ejemplo un gestor de impresoras que debe especificar el id de cada cliente.
- **Denominación Indirecta:** Los mensajes son enviados y recibidos a través de depósitos especializados llamados buzones. Es necesario disponer de servicios del sistema para el mantenimiento de los buzones (crear_buzon, eliminar_buzon)
 - La operación send deposita el mensaje producido en el buzón y la operación receive retira un mensaje del buzón y lo entrega al proceso receptor.

Copia

El intercambio de mensajes entre procesos transfiere el contenido del mensaje desde el espacio de direcciones del emisor al del receptor. Debe haber un compromiso entre seguridad y eficiencia

- **Copia todo el mensaje al espacio de direcciones del receptor:** Se transfiere el mensaje por valor. Es inevitable en sistemas distribuidos sin memoria común. Las modificaciones del emisor y el receptor sobre el mensaje se realizan sobre la copia local
 - **Problema:** Consume memoria y ciclos de procesador ya que pueden requerir que el mensaje sea copiado desde el espacio del emisor a un buffer del SO y desde allí al espacio del receptor. El SO debe disponer de un espacio de memoria dinámico para el proceso de copia
- ***Pasa un puntero al mensaje entre los dos procesos:*** Se transfiere el mensaje por referencia. Es la solución menos segura pero más rápida, proporciona al receptor una ventana al espacio de direcciones del emisor
 - **Problema:** Mientras el receptor este utilizando el mensaje el receptor no puede modificarlo.
- **Híbrido:** Inicialmente el SO intenta optimizar el rendimiento compartiendo una sola copia del mensaje que es accesible tanto al espacio de direcciones del emisor como al receptor siempre que ambos accedan solo para lectura. Si

alguno de ellos intenta modificar el mensaje el SO interviene y crea una copia física separada del mensaje para cada proceso.

Intercambio sincronizado o asíncrono de mensajes

- Sincronizado: El emisor y el receptor deben proceder juntos a completar la transferencia. La operación send es bloqueante, es decir, el emisor queda suspendido hasta que el receptor acepte el mensaje.
 - Ventaja: Facilidad de implementación, el emisor esta seguro de la recepción de su mensaje cuando prosigue la instrucción send.
 - Desventaja: Obligada operación sincronizada de emisores y receptores.

Intercambio síncrono o asíncrono de mensajes

- Asíncrono: El emisor no se bloque por receive pendientes. El SO almacena temporalmente los mensajes pendientes por receive. El emisor continua su ejecución independientemente de los receptores.
 - Problemas:
 - ◆ Posibilidad útil pero peligrosa si se abusa de ella pues un proceso pudiera generar tantos mensajes sin control y agotar la memoria temporal del sistema. Una solución es asignar un espacio de almacenamiento temporal para cada emisor/receptor.
 - ◆ Aplazamiento indefinido o Bloqueo entre procesos cuando se envía un mensaje y nunca se recibe por que el receptor se cancela o cuando el receptor espera por un mensaje que nunca se produce.

Otra solución pudiera ser receive sin bloqueo o con espera temporizada en este ultimo se especifica un límite de tiempo durante el cual debe completarse un intercambio de mensaje, sino llega ninguno el SO devuelve el control al receptor indicando que el plazo ha finalizado.

Longitud de los mensajes

- Debe haber un compromiso entre el recargo y la flexibilidad. No es de gran importancia la longitud en sistemas donde la transferencia se realiza mediante punteros pero cuando los mensajes son copiados debe ser cuidadosamente evaluado.
- Fija: Producen bajo recargo dado que los buffers del sistema serán también de tamaño fijo, lo que hace la asignación sencilla y eficaz.

- Problema: Los mensajes son naturalmente de tamaño variable y encajarlos en buffers de tamaño mas pequeño desperdician espacio y los mas largos deben ser divididos y enviados por partes.
- Variable: Crea buffers dinámicamente para que se ajusten al tamaño de cada mensaje individual.
- Problema: La gestión del fondo de memoria dinámica por parte del SO, lo cual es costos en términos de tiempo de CPU y también puede producir fragmentación de la memoria.

Protocolo de Exclusión Mutua con Paso de Mensajes

Receive (msj,...n) *Protocolo de Negociación*
Sección Crítica
 Send (msj,...,n) *Protocolo de Liberación*

Interbloqueos:

Dados:

- Un conjunto de procesos ejecutándose en un sistema
- Un conjunto de recursos que son utilizados por dichos procesos
 - Se dice que el conjunto de procesos se encuentra en un estado de interbloqueo cuando todos los procesos se encuentran esperando un recurso que mantiene retenido otro proceso del grupo.

En esa situación:

- Ningún proceso del grupo puede evolucionar (suspendido eternamente).
- Ningún proceso podrá obtener los recursos retenidos, puesto que no pueden ser liberados.

Se ha demostrado que las siguientes cuatro condiciones son necesarias (aunque no suficientes) para que se produzca un interbloqueo:

- Exclusión mutua: Al menos un recurso debe ser utilizado en exclusión mutua.
- Retener y esperar: Debe haber al menos un proceso que retenga un recurso y que haya pedido algún otro recurso que posea otro proceso, por lo que estará esperando.

- No expropiación: El sistema no puede arrebatar los recursos que ha asignado previamente a los procesos. Un proceso mantiene retenido un recurso hasta que deja de utilizarlo y lo libera voluntariamente.
- Espera circular: Debe existir un conjunto de procesos $\{P_1, P_2, \dots, P_i, \dots, P_n\}$ tal que P_1 se encuentra esperando un recurso que retiene P_2 , P_2 espera un recurso que retiene P_3 , ..., P_{n-1} espera un recurso que mantiene P_n y P_n espera un recurso que mantiene P_1 .

Tratamiento

- Emplear algún algoritmo o protocolo que asegure que nunca se va a poder producir un interbloqueo. Esta solución puede adoptar dos formas alternativas:
 - Prevención: consiste en conseguir que no puedan darse alguna de las cuatro condiciones de Coffman. De esta forma, el interbloqueo no puede llegar a producirse.
 - Evitación: consiste en llevar la cuenta de los recursos disponibles en el sistema, los recursos que poseen los procesos y los que pueden llegar a solicitar. Cada vez que se hace una petición de un recurso, el sistema analiza toda esa información para conceder (o denegar) dicho recurso.
- Utilizar un algoritmo que pueda detectar una situación de interbloqueo (detección) y seguir alguna técnica que permita deshacer dicha situación (recuperación).
- Ignorar el problema (Ostrich algorithm), asumiendo que dicha situación nunca se dará en el sistema. Es la aproximación que mantienen muchos sistemas operativos, incluido Unix.

Métodos para tratar los interbloqueos

- Denegación de inicio de proceso
 - Evita el inicio de procesos
 - Se impide que un proceso se inicie si los recursos máximos necesarios son superiores a los disponibles
 - Incluso aunque...
 - ◆ No hubiera llegado a usarlos todos
 - ◆ Pueda ir avanzando con los disponibles

- Denegación de asignación de recurso
 - Evita la asignación de recursos
 - Se impide que un recurso se asigne si esto llevaría a un estado inseguro
 - Se emplea el algoritmo del banquero

Estado seguro

Un estado (de asignación de recursos) se considera cuando en él no hay posibilidad de interbloqueo

Para que un estado sea seguro, es necesario que exista al menos una secuencia segura

Secuencia segura

Es una cierta ordenación de los procesos que asegura se podrá completar la ejecución de todos ellos

- Al menos uno puede ejecutarse, y al acabar liberará recursos, que utilizará otro proceso... y así sucesivamente
- Es una visión pesimista, asume que todos los procesos necesitarán de todos los recursos al mismo tiempo
- Si dicha secuencia existe, como mucho cada proceso tendrá que esperar a que los procesos que van antes que él en la secuencia, liberen sus recursos

Algoritmo del banquero

Cuando un proceso realice una petición de recursos, el sistema se los concederá sólo en el caso de que la petición mantenga al sistema en un estado seguro

En dicha petición se comprueba que...

1. El proceso no pida más recursos de los que originalmente necesitaba
2. Que el recurso pedido está disponible
3. Que tras la asignación el sistema queda en estado seguro

Tema III- Gestión de Memoria Contigua y Virtual

Memoria Contigua

Es un método de asignación de memoria en el que los bloques de memoria se asignan en un trozo único y continuo. Este método se utiliza normalmente cuando es necesario almacenar grandes cantidades de datos en la memoria, o cuando se debe acceder a los datos de forma secuencial.

Ventajas

- Es muy eficiente para los datos a los cuales se deben acceder de forma secuencial, porque los datos están almacenados de forma lineal.
- Es más fácil hacer un seguimiento de un solo bloque de memoria que de múltiples bloques dispersos.
- Se puede utilizar para crear "vistas" de los datos, permitiendo acceder a diferentes partes de los datos sin tener que copiar todo el conjunto de datos en la memoria.

Desventajas

- Puede ser difícil encontrar un bloque suficientemente grande de memoria contigua para asignar, especialmente en sistemas con una gran cantidad de memoria.
- Puede ser difícil desasignar un bloque de memoria contigua, ya que todo el bloque debe ser liberado a la vez.
- Si es necesario acceder a los datos de un bloque contiguo de forma no secuencial, puede ser más lento que si los datos se almacenan de forma no contigua.

Asignación Contigua

Cada objeto lógico es colocado en un conjunto de posiciones de memoria con direcciones estrictamente consecutivas. Un método habitual consiste en particionar la memoria física disponible y satisfacer las solicitudes de memoria asignando particiones libres adecuadas, si las hay. Estas particiones pueden ser definidas estática o dinámicamente

Asignación no contigua

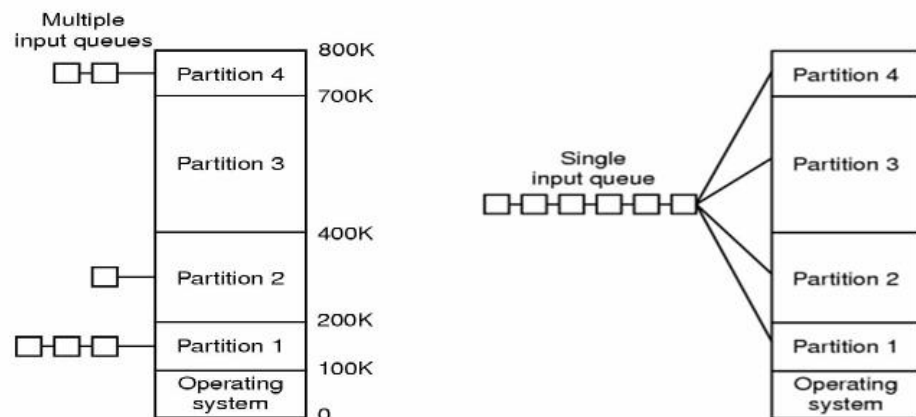
La memoria se asigna de tal modo que partes de un sólo objeto lógico pueden ser colocadas en áreas no contiguas de memoria física

Particionamiento Estático: Una cola o múltiples colas (Primer ajuste, siguiente ajuste, mejor ajuste, peor ajuste)

La división de la memoria (número y tamaño de las particiones) permanece fija y se efectúa durante el proceso de generación del sistema, antes de ejecutar los programas de usuario. Se toma en cuenta la capacidad de la memoria física disponible, el grado deseado de multiprogramación y los tamaños típicos de los procesos frecuentemente ejecutados en la instalación.

Es un método adecuado para entornos estáticos en donde la carga de trabajo es predecible y sus características son conocidas. Ejemplo: entornos de producción en un ambiente bancario, con poco o ningún desarrollo.

Particionamiento Estático



Desventajas de los esquemas

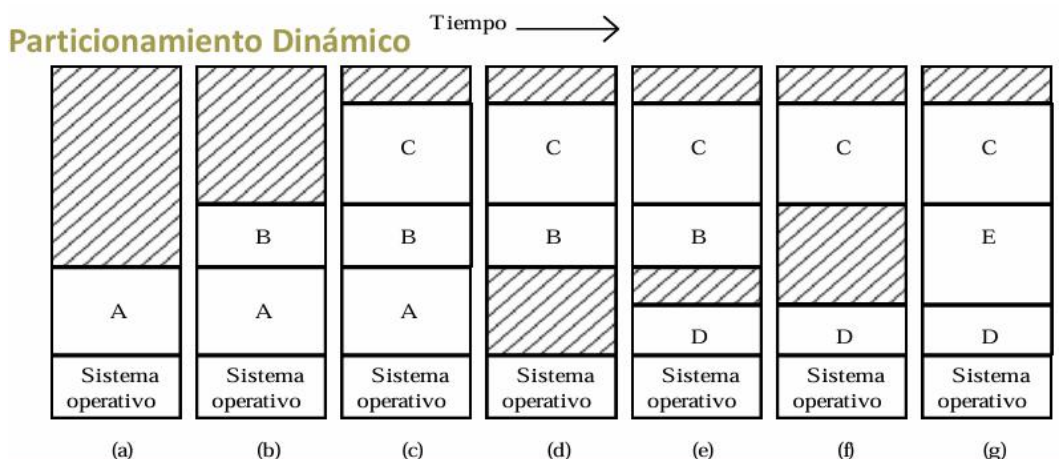
- Puede suceder que la cola asociada a una partición esté vacía pero la cola de una partición pequeña esté completamente ocupada.
- Existen varias formas de atender las peticiones:
 - Cada vez que se libera una partición se podría cargar y ejecutar en ellas la tarea que esté de primera en la cola y con tamaño adecuado (primer ajuste)
 - Como no es deseable que se desperdicie una partición de gran tamaño en una tarea pequeña, otra estrategia consiste en buscar en toda la cola de entrada el tamaño que mejor ajuste. En general, las tareas pequeñas son discriminadas para lo cual se puede definir una partición pequeña.
 - Imponer un límite de tiempo para la ejecución de los procesos.

Desventajas del Particionamiento Estático

- Es inflexible e incapaz de adaptarse a las necesidades cambiantes del sistema

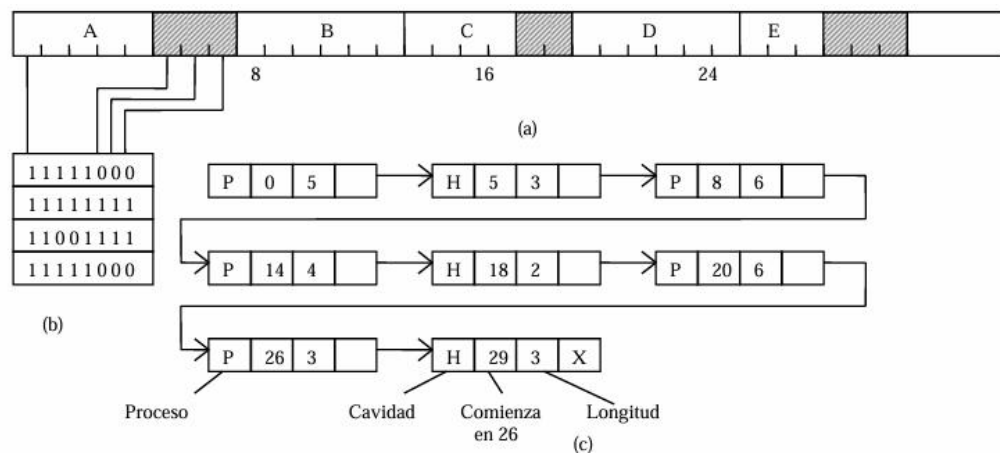
- Particionamiento Dinámico:** Mapas de Bits, Listas enlazadas: Primer Ajuste, Mejor Ajuste y Peor Ajuste, Sistema de los asociados o compañeros.

- Es una asignación adaptativa de la memoria
- Se presenta como una respuesta a demandas de usuario
- Cada partición tiene el tamaño justo necesario para satisfacer la petición de espacio de un objeto (programa o un área de datos compartida)
- Es un método adecuado para entornos de desarrollo de programas o donde la carga de trabajo es impredecible 🌀
- En general existen tres formas utilizadas por los SO para administrar el uso de la memoria:
 - Mapas de Bits
 - Listas enlazadas: Primer Ajuste, Mejor Ajuste y Peor Ajuste
 - Sistema de los asociados o compañeros



Particionamiento Dinámico: Mapas de Bits

- La memoria se divide en unidades de asignación que pueden ser tan pequeñas como unas cuantas palabras o tan grande como unos cuantos Kb
- A cada unidad de asignación le corresponde un bit en el mapa de bits, el cual toma el valor 0 si la unidad está libre y 1 si está ocupado
- Mientras más pequeña sea la unidad de asignación más grande será el mapa de bits, si la unidad de asignación es grande el mapa de bits será pequeño pero habrá fragmentación interna
- Cuando se requiere un espacio de k unidades, el administrador de la memoria debe recorrer el mapa para encontrar una cadena de k ceros consecutivos



Particionamiento Dinámico: Listas Enlazadas

- Primer ajuste: el administrador rastrea la lista hasta hallar una cavidad que sea lo suficientemente grande
- Siguiendo ajuste: igual que el primer ajuste pero empezando en donde se quedó la vez anterior
- El que mejor ajusta: busca en toda la lista hasta encontrar la cavidad que ajusta mejor
- El que peor ajusta: toma siempre la cavidad más grande disponible

Particionamiento Dinámico: Sistema de los Asociados o Compañeros

Memoria											Cavidades						
	128k		256k		384k		512k		640k		768k		896k		1M		
Inicialmente																	

Desventajas del Particionamiento Dinámico

- La gestión de memoria es más compleja que en el particionamiento estático, consumiendo más espacio y tiempo de uso del SO
- Presenta poca fragmentación interna cuando se usa el sistema de los Asociados o Compañeros
- Presenta fragmentación externa, es decir, se desaprovecha memoria entre las particiones, imponiendo una penalidad de tiempo para la compactación

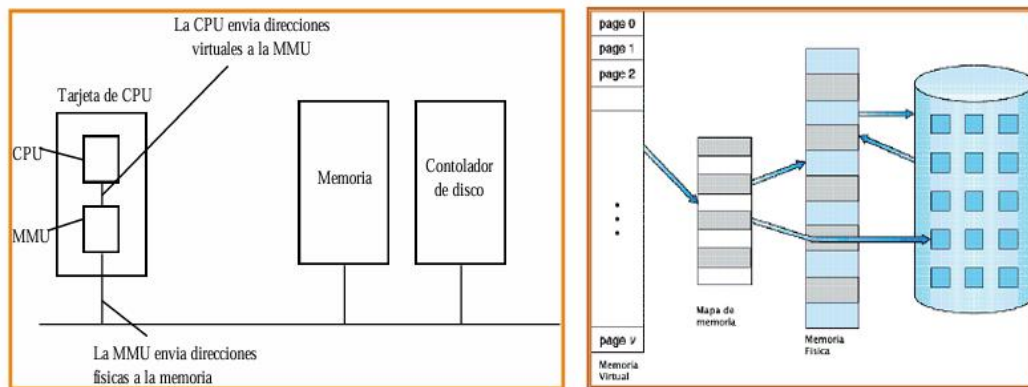
La Compactación

- Sólo es viable si se usa reubicación dinámica (cambian los espacios físicos de los procesos)
- Puede consumir mucho tiempo (hay que mover bloques de memoria)

No Contigua: Memoria Virtual

Es un esquema de gestión de memoria en donde puede que sólo una parte del espacio de direcciones virtuales de un proceso “residente” sea cargada realmente en memoria física.

El tamaño permisible del espacio de direcciones virtuales de un solo proceso puede exceder a la capacidad máxima de la memoria física instalada en un sistema dado. Esto se consigue manteniendo una imagen del espacio de direcciones virtuales completo de un proceso en memoria secundaria y trayendo a memoria principal partes de esa imagen cuando sean necesarias



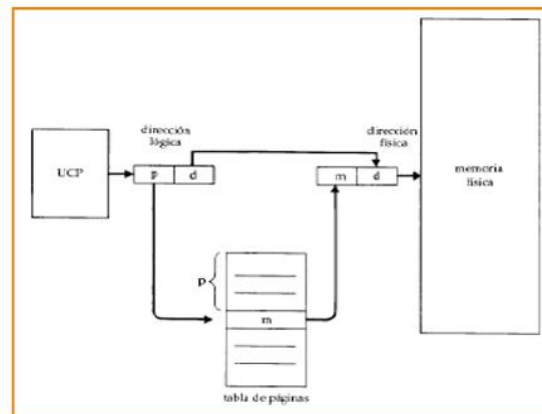
Paginación: Esquema de memoria, Tablas, Direccionamiento, TLB

- **TMP/TP** Tabla de Páginas: Una por cada proceso activo. Proporciona información sobre la Ausencia/Presencia de una página en memoria principal
- **TMM** Tabla de Mapa de Memoria: Se usa para llevar la cuenta de las zonas de memoria física que están disponibles para asignación
- **TMA** Tabla de Mapa de Archivo: Existe una por cada proceso activo. Contiene direcciones de almacenamiento secundario de todas las páginas. Se utiliza para cargar en memoria principal los elementos ausentes. Su base puede ser almacenada en la PCB de los procesos

Paginación

Cada dirección generada por la CPU es una par ordenado (p,d) , donde:

- **p** es el número de página o índice de la tabla de páginas
- **d** es el desplazamiento dentro de la página



Estructura de la Tabla de Páginas (TP)

- Denominador común
 - Una tabla de páginas para cada proceso
- ¿Cómo localiza el SO la TP de un proceso?
 - En la PCB

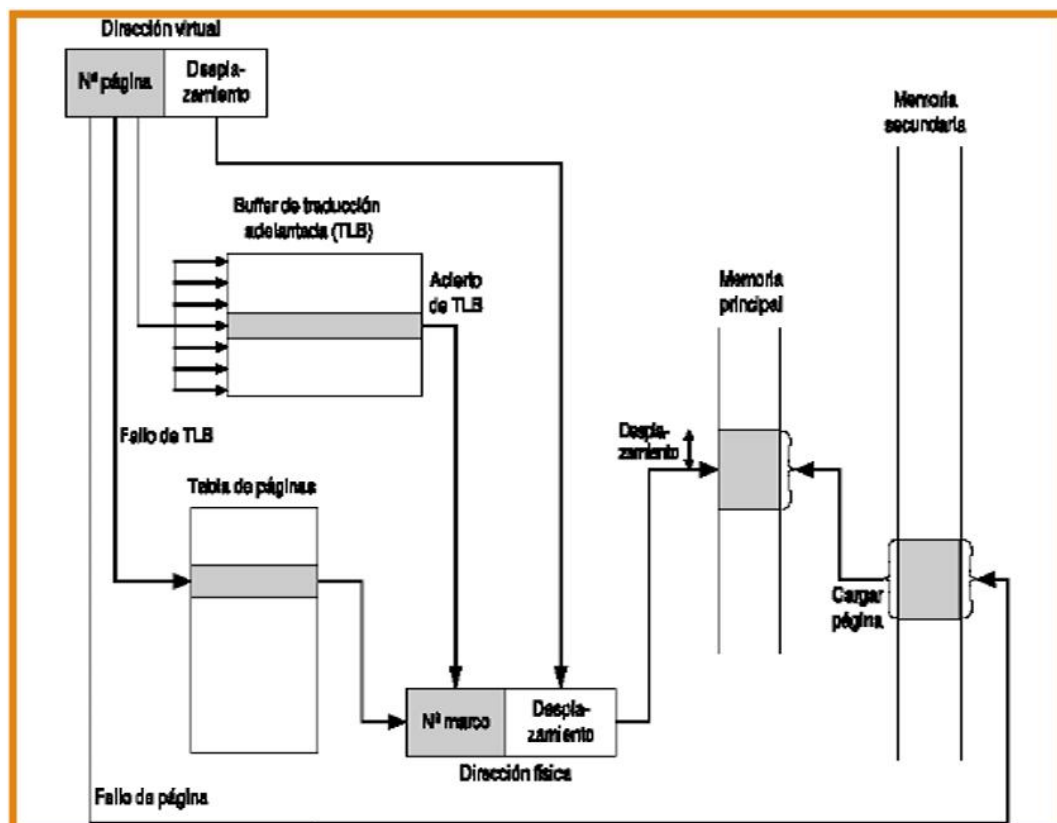
- Contador de instrucciones, registros, información de E/S, etc.
- Puntero a la TP
- ¿Qué ocurre en un cambio de contexto?
 - El despachador cargará los registros con los valores del nuevo proceso
 - A partir de la TP almacenada, cargará los valores correctos de la TP en hw

Aspectos de Diseño TLB (Translation Lookahead Buffer, o tabla de registros asociativos)

Pequeño caché especial en hardware para el que cada registro se compone de dos partes: clave y valor.

Funcionamiento

- Se presenta una clave y, si encuentra alguna coincidencia, devuelve el valor correspondiente
- Permite búsquedas rápidas pero el hardware es costoso



Funcionamiento: acceso a la posición i

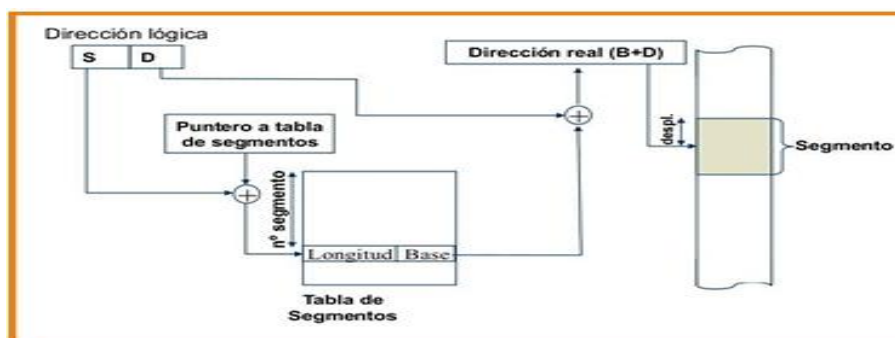
- Obtiene el número de página donde se encuentra i .
- Si está en TLB $\Rightarrow$ Obtenemos el marco de página donde se encuentra
- Sino, acceso a la TMP y actualiza la TLB
- Si TLB llena $\Rightarrow$ Sustitución de una de las existentes
 - Ojo, cambio de contexto
- Desalojar (borrar) el TLB

Tasa de aciertos

- Porcentaje de las veces que un número de página se encuentra en los registros asociativos
- Buenas tasas de aciertos: 80% - 98% (Regla de dedo)
- Ejemplo: Intel 80486 $\Rightarrow$ TLB de 32 entradas. Fabricantes reportan una tasa de acierto del 98%

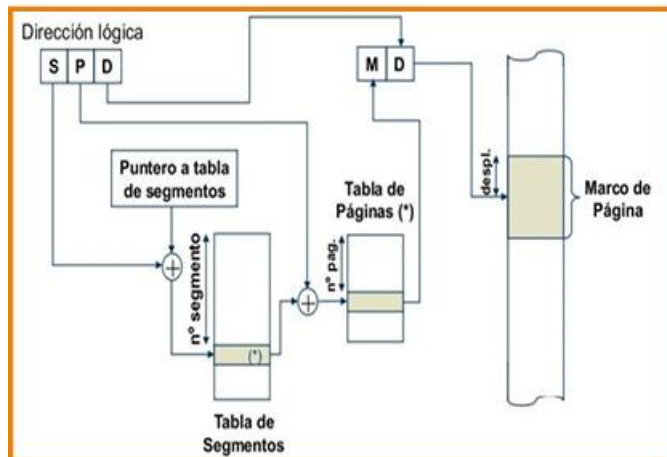
Segmentación: Esquema de memoria, Tablas, Direccionamiento, TLB

- Es un esquema de manejo de memoria mediante el cual la estructura del programa refleja su división lógica; llevándose a cabo una agrupación lógica de la información en bloques de tamaño variable denominados segmentos
- Cada uno de ellos tiene información lógica del programa: texto, datos, pila, entre otros
- **Enlace dinámico entre segmentos:** Puede evitarse realizar el enlace antes de comenzar a ejecutar un programa, estableciéndolos solo cuando sea necesario



Paginación con Segmentación: Esquema de memoria, Tablas, Direccionamiento, TLB

Es una combinación de segmentación y paginación para obtener las ventajas de ambas. En lugar de tratar un segmento como una unidad contigua, este puede dividirse en páginas. Cada segmento puede ser descrito por su propia tabla de páginas



Algoritmos de Remoción de páginas

Óptimo, LRU, FIFO, Reloj, Segunda Oportunidad, NRU, Working Set, WS Clock.

Óptimo

Definición

Este algoritmo debe generar el mínimo número de fallos de página. Por ello, la página que se debe reemplazar es aquella que tardará más tiempo en volverse a usar.

Problemática

Es muy sencillo de entender teóricamente, pero imposible de implementar, dado que requiere conocer a futuro las necesidades del sistema.

FIFO

Definición

En este algoritmo, el sistema operativo lleva un registro de todas las páginas de la memoria en una cola, la página más antigua se encuentra al principio de la cola. Cuando es necesario reemplazar una página, se selecciona la página que se encuentra al principio de la cola para eliminarla.

Existen 3 casos:

- ***El primero:*** Ocurre cuando el conjunto puede almacenar la nueva página entrante y la página aún no está presente en el conjunto. En este caso, normalmente almacenamos la nueva página en el conjunto.
- ***El segundo:*** Este sucede cuando el conjunto puede almacenar la nueva página entrante, pero la página ya está presente en el conjunto. En este caso, la marcamos como una visita a la página y no aumentaremos el recuento de la variable FallasPagina.
- ***El tercer:*** Este ultimo pasa cuando el conjunto está lleno. En este caso, debemos realizar el método FIFO para eliminar páginas de la cola.

LRU

Definición

Es un algoritmo voraz donde la página que se reemplazará es la utilizada menos recientemente. La idea se basa en la localidad de referencia, la página utilizada menos recientemente no es probable.

Implementación

Requiere un apoyo del hardware, la solución sera agregar un contador a cada marco, tener una constante actualización cuando se haga una referencia a dicha pagina y elegir como victima a la pagina con menor conteo. Su principal desventaja es que se debe recorrer cada contador para determinar cual marco usar, si usamos 100 marcos, haría que el proceso de cambiar de pagina, sea más lento.

Reloj

Definición

Tiene una lista circular, de forma que al llegar al último elemento de la lista, pasa automáticamente al primero. Los elementos no se mueven al final de la cola cuando son accedidos, simplemente se pone su bit de referencia a 1. Esto nos evita tener que hacer movimientos de punteros en el caso de implementarlo con una lista enlazada. De hecho, se puede implementar con un array perfectamente, ahorrando así memoria.

Segunda Oportunidad

Definición

Funciona mirando el frente de la cola como lo hace FIFO, pero en lugar de paginar inmediatamente esa página, verifica si su bit de referencia está configurado. Si no está configurado, la página se intercambia. De lo contrario, el bit de referencia se borra, la página se inserta al final de la cola (como si fuera una página nueva) y este proceso se repite. Esto también puede considerarse como una cola circular. Si todas las páginas tienen su bit de referencia configurado, en el segundo encuentro de la primera página en la lista, esa página se intercambiará, ya que ahora tiene su bit de referencia borrado. Si todas las páginas tienen su bit de referencia borrado, entonces el algoritmo de segunda oportunidad degenera en FIFO puro.

NRU

Definición

Es el algoritmo de segunda oportunidad mejorado, se asocia a cada página un bit de referencia y uno de modificación. El bit de referencia se actualiza cada vez que se opera sobre la página (lectura o escritura). El bit de modificación se activa cada vez que se escribe. Cada cierto tiempo, se genera una interrupción para limpiar el bit de referencia.

Las páginas se dividen en cuatro clases:

- Clase 0: no referenciada, no modificada.
- Clase 1: no referenciada, modificada.
- Clase 2: referenciada, no modificada.
- Clase 3: referenciada, modificada.

Working set

Definición

Determina el número de páginas únicas en una localidad y mantiene ese número de marcos para el proceso. Por lo tanto, todas las páginas requeridas pueden estar en la memoria principal para una determinada localidad. Esto previene el thrashing y reduce las fallas de página. A medida que la localidad cambia, también se modifica el número de marcos asignados.

WSClock

Definición

El algoritmo parte de Working Set (WS) y el algoritmo reloj, es un nuevo mecanismo de control de carga para el acceso a la memoria auxiliar. El nuevo algoritmo combina la característica más útil de WS—un control de carga natural y efectivo que previene el thrashing—con la simplicidad y eficiencia de CLOCK.

Envejecimiento

Definición

Como una mejora de NFU, se puede introducir un parámetro de olvido (o envejecimiento) sobre las referencias pasadas. Un mecanismo para implementar el envejecimiento, consiste en desplazar los contadores de un bit a la derecha, en cada tick del reloj introduciendo el bit R por la izquierda. De esta forma una página referenciada durante el último tick permanecerá en memoria sobre cualquiera que no lo haya sido

Otros Tópicos

La anomalía de Belady

Es el fenómeno en el que el aumento del número de marcos de página da como resultado un aumento del número de errores de página para ciertos patrones de acceso a la memoria. Este fenómeno se experimenta comúnmente cuando se utiliza el algoritmo de reemplazo de páginas primero en entrar, primero en salir (FIFO). En FIFO, el error de página puede aumentar o no a medida que aumentan los marcos de página, pero en algoritmos óptimos y basados en pila como LRU, a medida que aumentan los marcos de página, el error de página disminuye. László Bélády lo demostró en 1969.

Determinación del Tamaño de la página

Hemos visto que un tamaño de tabla de páginas más grande causa un sobre costo adicional porque debemos dividir esa tabla en páginas y luego almacenarla en la memoria principal.

Nuestra preocupación debe centrarse en la ejecución de los procesos, no en la ejecución de la tabla de páginas. La tabla de páginas proporciona un soporte para la ejecución del proceso. Cuanto más grande sea la tabla de páginas, mayor será el sobre costo.

Ahora tomando en cuenta lo anterior mencionado:

Tamaño de la Tabla de Páginas = número de entradas de página en la tabla de páginas X tamaño de una entrada de página

Consideremos un ejemplo:

Espacio de Direcciones Virtuales = 2 GB = 2×2^{30} Bytes

Tamaño de la Página = 2 KB = 2×2^{10} Bytes

Número de Páginas en la Tabla de Páginas = $(2 \times 2^{30}) / (2 \times 2^{10}) = 1$ millón de páginas

Habría 1 millón de páginas, lo cual es un número bastante grande. Sin embargo, intentemos hacer que el tamaño de la página sea más grande, digamos 2 MB.

Entonces, el número de páginas en la tabla de páginas sería:

Número de páginas en la tabla de páginas = $(2 \times 2^{30}) / (2 \times 2^{20}) = 1$ K páginas.
Si comparamos los dos escenarios, podemos observar que el tamaño de la tabla de páginas es inversamente proporcional al tamaño de la página.

En la paginación, siempre hay un desperdicio en la última página. Si el espacio de direcciones virtuales no es un múltiplo del tamaño de la página, quedarán algunos bytes restantes, y tendremos que asignar una página completa para esos bytes. Esto simplemente es un sobre costo.

Consideremos lo siguiente:

Tamaño de la página = 2 KB

Espacio de direcciones virtuales = 17 KB

Entonces, el número de páginas se calcularía como:

Número de páginas = $17 \text{ KB} / 2 \text{ KB} = 8.5$

Como no podemos tener media página, redondeamos hacia arriba, lo que nos da 9 páginas. Sin embargo, la novena página solo contendrá 1 byte, y el resto de la página (es decir, 2 KB - 1 byte) se desperdiciará. Esto es un ejemplo de cómo la paginación puede generar fragmentación interna o desperdicio de memoria.

En general, si:

Tamaño de la página = p bytes

Tamaño de la entrada = e bytes

Espacio de direcciones virtuales = S bytes

Entonces, el sobre costo O se puede calcular como:

$O = (S/p) \times e + (p/2)$

En promedio, el número de páginas desperdiciadas en un espacio virtual es $p/2$ (la mitad del número total de páginas).

Para minimizar el sobre costo, derivamos la función de sobre costo con respecto a p e igualamos a cero:

$$\partial O / \partial p = 0$$

$$-S/(p^2) + 1/2 = 0$$

Resolviendo para p , obtenemos:

$$p = \sqrt{2 \cdot S \cdot e} \text{ bytes}$$

Por lo tanto, si el tamaño de la página es $\sqrt{2 \cdot S \cdot e}$ bytes, el sobre costo será mínimo.

Protección de la memoria cuando se utiliza la paginación y la segmentación

¿Qué es la protección de paginación?

El proceso de paginación debe protegerse utilizando el concepto de inserción de un bit adicional llamado bit válido/inválido. La protección de la memoria en la paginación se logra asociando bits de protección con cada página. Estos bits están asociados con cada entrada de la tabla de páginas y especifican la protección en la página correspondiente.

¿Qué es la protección de segmentación?

La segmentación de memoria también permite establecer mecanismos de protección para garantizar que un programa no pueda acceder a segmentos de memoria que no le corresponden. Esto ayuda a prevenir errores y garantiza la integridad del sistema.

Compartición de la memoria cuando se utiliza paginación o segmentación.

Paginacion

Código compartido

Una copia de sólo lectura del código compartido entre procesos (p. ej., editores de textos, compiladores, sistemas de ventana)

El código compartido debe aparecer en la misma posición en el espacio de dirección lógico de todos los procesos

Segmentación

Dos o más procesos pueden compartir un mismo segmento, bajo reglas de protección; aunque no sean propietarios de los mismos.

Políticas

Lectura, Escritura, Ubicación, Gestión del Conjunto residente, Asignación, Reemplazo, Estrategias combinadas asignación/reemplazo, Vaciado

Lectura

- Paginación por Demanda
- Paginación Previa

Políticas de Ubicación

- Para sistemas con segmentación pura
- Para paginación y segmentación/paginada

Políticas de Reemplazo

- Alcance Global
- Alcance Local
- Mecanismos de Reemplazo: Algoritmos de Sustitución

Políticas de Asignación

- Asignación Equitativa
- Asignación Proporcional

Gestión del Conjunto Residente

- Asignación Fija con Alcance Global
- Asignación Fija con Alcance Local
- Asignación Variable con Alcance Global
- Asignación Variable con Alcance Local

Políticas de Vaciado

- Vaciado por Demanda
- Vaciado Previo

Estrategia del Conjunto de Trabajo

- Working Set

Frecuencia de Fallo de Página

Políticas de Control de Carga

- Suspensión de Procesos

Políticas de Lectura

Determina cuando una página debe traerse a memoria principal

- Por Demanda
 - Se trae una página a memoria principal sólo cuando se hace referencia a una posición en dicha página
 - Produce muchos fallos de página cuando un proceso se ejecuta por primera vez; pero debe disminuir a medida que se traigan a memoria más páginas (principio de cercanía)
- Previa
 - Se cargan otras páginas distintas a la demandadas, debido a un fallo de página
 - El principio de cercanía sugiere que sea más eficaz traer páginas que residen contiguamente en disco
 - Eficiencia no establecida definitivamente cuando las páginas extras traídas, no son “frecuentemente” referenciadas

Políticas de Ubicación

Determina dónde va a residir una parte de un proceso en memoria real

- Ubicación en sistemas con segmentación pura:
 - Primer-ajuste, próximo-ajuste, mejor-ajuste (relevante)

- Ubicación en sistemas con paginación o segmentación/paginada:
 - El hardware decide dónde poner la página (no relevante)

Políticas de Reemplazo

Trata de la selección de la página a reemplazar en memoria principal, al momento de cargar una nueva página. Esto ocurre siempre que la memoria principal esté llena (ningún marco libre disponible)

- Ocurre frecuentemente debido a que el SO intenta traer a memoria principal tantos procesos como pueda, para aumentar el nivel de multiprogramación
- Algunos marcos que están bloqueados no pueden paginarse (paged out):
 - Mucho del kernel es mantenido en marcos bloqueados; así como también, las estructuras clave de control y buffers de I/O
- Se desea reemplazar la página que tenga menos posibilidad de ser referenciada en otro momento
- Se debe evitar la complejidad de la política de reemplazo para no producir overhead del SO
- El SO decide, entre el conjunto de páginas consideradas, si el reemplazo tiene un:
 - Alcance Local: Limitado al proceso que ha sufrido un fallo de página
 - Alcance Global: El conjunto de todas las páginas en marcos no bloqueados (unlocked)

Gestión de Memoria en UNIX y Windows

Windows 3.x y Windows 9x

La paginación ha sido una característica de Microsoft Windows desde Windows 3.0 en 1990. Windows 3.x crea un archivo oculto llamado 386SPART.PAR o WIN386.SWP para usar como un archivo de intercambio. Generalmente se encuentra en el directorio raíz, pero puede aparecer en otro lugar (normalmente en el directorio WINDOWS). Su tamaño depende de cuánto espacio de intercambio tenga el sistema (una configuración seleccionada por el usuario en Panel de control → Mejorado en "Memoria virtual"). Si el usuario mueve o elimina este archivo, aparecerá una pantalla azul la próxima vez que inicie Windows, con el mensaje de error "El archivo de intercambio permanente está dañado". Se le pedirá al usuario que elija si eliminar o no el archivo (incluso si no existe).

Windows 95, Windows 98 y Windows, utilizan un archivo similar y su configuración se encuentra en Panel de control → Sistema → pestaña Rendimiento → Memoria virtual. Windows configura automáticamente el tamaño del archivo de la página para comenzar en 1,5 veces el tamaño de la memoria física y expandir hasta 3 veces la memoria física si es necesario. Si un usuario ejecuta aplicaciones que hacen un uso intensivo de la memoria en un sistema con poca memoria física, es preferible establecer manualmente estos tamaños en un valor superior al predeterminado.

Windows NT

El archivo utilizado para la paginación en la familia de Windows NT es pagefile.sys. La ubicación predeterminada del archivo de página es el directorio raíz de la partición donde está instalado Windows. Windows se puede configurar para usar espacio libre en cualquier unidad disponible para archivos de página. Sin embargo, se requiere que la partición de arranque (es decir, la unidad que contiene el directorio de Windows) tenga un archivo de página si el sistema está configurado para escribir volcados de kernel o de memoria completa después de una pantalla azul de la muerte. Windows usa el archivo de paginación como almacenamiento temporal para el volcado de memoria. Cuando se reinicia el sistema, Windows copia el volcado de memoria del archivo de página a un archivo separado y libera el espacio que se usó en el archivo de página.

Unix y sistemas similares a Unix

Los sistemas Unix y otros sistemas operativos similares a Unix utilizan el término "intercambiar" para describir el acto de sustituir espacio en disco por RAM cuando la RAM física está llena. En algunos de esos sistemas, es común dedicar una partición completa de un disco duro al intercambio. Estas particiones se denominan particiones de intercambio. Muchos sistemas tienen un disco duro completo dedicado al intercambio, separado de las unidades de datos, que contiene solo una partición de intercambio. Un disco duro dedicado al intercambio se denomina "unidad de intercambio" o una "unidad de memoria virtual" o un "disco de memoria virtual". Algunos de esos sistemas solo admiten el intercambio a una partición de intercambio; otros también admiten el intercambio de archivos.

Linux

El kernel de Linux admite una cantidad prácticamente ilimitada de backends de intercambio (dispositivos o archivos) y también admite la asignación de prioridades de backend. Cuando el kernel intercambia páginas fuera de la memoria física, utiliza el backend de mayor prioridad con espacio libre disponible. Si se asigna la misma prioridad a varios backends de intercambio, se utilizan de forma rotatoria (que es algo similar a los diseños de almacenamiento RAID 0), lo que proporciona un rendimiento mejorado siempre que se pueda acceder de manera eficiente a los dispositivos subyacentes en paralelo.

Intercambiar archivos y particiones

Desde la perspectiva del usuario final, los archivos de intercambio en las versiones 2.6.xy posteriores del kernel de Linux son prácticamente tan rápidos como las particiones de intercambio; la limitación es que los archivos de intercambio deben asignarse de forma contigua en sus sistemas de archivos subyacentes. Para aumentar el rendimiento de los archivos de intercambio, el núcleo mantiene un mapa de dónde se colocan en los dispositivos subyacentes y accede a ellos directamente, sin pasar por el caché y evitando la sobrecarga del sistema de archivos. Cuando residen en HDD, que son dispositivos de medios magnéticos rotativos, una de las ventajas de usar particiones de intercambio es la capacidad de colocarlas en áreas de HDD contiguas que proporcionan un mayor rendimiento de datos o un tiempo de búsqueda más rápido. Sin embargo, la flexibilidad administrativa de los archivos de intercambio puede superar ciertas ventajas de las particiones de intercambio. Por ejemplo, un archivo de intercambio se puede colocar en cualquier sistema de archivos montado, se puede configurar en cualquier tamaño deseado y se puede agregar o cambiar según sea necesario. Las particiones de intercambio no son tan flexibles; no se pueden ampliar sin utilizar herramientas de gestión de volumen o partición, que introducen diversas complejidades y posibles tiempos de inactividad.

Intercambio

Swappiness es un parámetro del kernel de Linux que controla el peso relativo otorgado al intercambio de memoria en tiempo de ejecución, en lugar de eliminar páginas del caché de páginas del sistema, siempre que no se pueda cumplir con una solicitud de asignación de memoria de forma gratuita. La capacidad de intercambio se puede establecer en un valor de 0 a 200. Un valor bajo hace que el kernel prefiera desalojar páginas del caché de páginas, mientras que un valor más alto hace que el kernel prefiera intercambiar "frío" páginas de memoria. El valor predeterminado es 60; configurarlo más alto puede causar una latencia alta si las páginas inactivas deben volver a intercambiarse (al interactuar con un programa que había estado inactivo, por ejemplo), mientras que configurarlo más bajo (incluso 0) puede causar una latencia alta cuando los archivos que habían sido desalojados del el caché debe leerse nuevamente, pero hará que los programas interactivos respondan mejor, ya que será menos probable que necesiten intercambiar páginas frías. El intercambio también puede ralentizar aún más los discos duros porque implica muchas escrituras aleatorias, mientras que los SSD no tienen este problema. Ciertamente, los valores predeterminados funcionan bien en la mayoría de las cargas de trabajo, pero los escritorios y los sistemas interactivos para cualquier tarea esperada pueden querer reducir la configuración, mientras que el procesamiento por lotes y los sistemas menos interactivos pueden querer aumentarla.

Intercambio de muerte

Cuando la memoria del sistema es muy insuficiente para las tareas actuales y una gran parte de la actividad de la memoria pasa por un intercambio lento, el sistema

puede volverse prácticamente incapaz de ejecutar ninguna tarea, incluso si la CPU está inactiva. Cuando todos los procesos están esperando el intercambio, se considera que el sistema está en muerte de intercambio.

La muerte del intercambio puede ocurrir debido a una sobreasignación de memoria configurada incorrectamente.

La descripción original del "intercambio de muerte" el problema se relaciona con el servidor X. Si el código o los datos utilizados por el servidor X para responder a una pulsación de tecla no están en la memoria principal, entonces, si el usuario pulsa una tecla, el servidor tomará una o más fallas de página, lo que requerirá que esas páginas se lean desde el intercambio antes de que se pueda ejecutar la pulsación de tecla. procesada, ralentizando la respuesta a la misma. Si esas páginas no permanecen en la memoria, tendrán que volver a activarse para manejar la siguiente pulsación de tecla, lo que hará que el sistema prácticamente no responda, incluso si en realidad está ejecutando otras tareas con normalidad.

MacOS

macOS usa varios archivos de intercambio. La instalación predeterminada (y recomendada por Apple) los coloca en la partición raíz, aunque es posible colocarlos en una partición o dispositivo separado.

Parte Práctica

1. Diagrama de 5 y 7 estados de los procesos

2. Ejercicios de Diagramas de Gantt para los distintos algoritmos de planificación y calcular su eficiencia: Tiempos promedio de espera, ejecución, cola.

3. Ejercicios para determinar si cumplen las 4 condiciones de la exclusión mutua

4. Ejercicios de programación concurrente a ser resueltos con semáforos, monitores o primitivas de comunicación Send y Receive

5. Ejercicios Productor/Consumidor o Lectores/Escritores

6. Ejercicios del Algoritmo del Banquero y grafos para interbloqueos

7. Esquema de Memoria: Paginación, Segmentación, Segmentación paginada

8. Dado un tamaño de memoria y una secuencia de accesos, elabora el diagrama de ejecución para algoritmos de remoción de páginas: Óptimo, LRU, FIFO, Reloj, Segunda Oportunidad, NRU, Working Set, WS Clock.

9. Cálculo del tamaño de memoria, tabla de páginas, dirección virtual y real, máximo direccionamiento, etc.